

Ανάλυση Δεδομένων 2025-26

4η Εργασία – Ενιαίο Explanatory Report

Δέντρα ταξινόμησης και ensemble μέθοδοι για πρόβλεψη
τύπου κρασιού

Ρακοβαλής Παύλος
ΑΕΜ: 6931

11 Μαΐου 2026

Περίληψη

Το παρόν αρχείο είναι το ενιαίο επεξηγηματικό report της 4ης εργασίας. Είναι γραμμένο με εκπαιδευτικό ύφος, σαν να απευθύνεται σε κάποιον που δεν έχει προηγούμενη εμπειρία στην Ανάλυση Δεδομένων. Για κάθε ερώτημα παρουσιάζεται πρώτα τι ζητάει η εκφώνηση, στη συνέχεια δίνεται το απαραίτητο θεωρητικό υπόβαθρο και τέλος εξηγείται βήμα προς βήμα η λύση και η ερμηνεία των αποτελεσμάτων. Σε αυτή την έκδοση έχουν ενσωματωθεί τα Ερωτήματα 1, 2, 3, 4 και 5. Τα επόμενα ερωτήματα θα προστεθούν στο ίδιο αρχείο.

Περιεχόμενα

1 Γενικό πλαίσιο της εργασίας	4
1.1 Προβλεπτικές μεταβλητές	4
1.2 Training set και test set	4
1.3 Πώς μετράμε την απόδοση	5
2 Ερώτημα 1: Δέντρο ταξινόμησης μέγιστου βάθους 2	5
2.1 Τι ζητάει το ερώτημα	5
2.2 Θεωρητικό υπόβαθρο	5
2.3 Πώς αποφασίζει το δέντρο ποιες μεταβλητές θα χρησιμοποιήσει	6
2.4 Πώς επιλέγεται το threshold μίας μεταβλητής	6
2.5 Γιατί επιλέχθηκε πρώτα το total sulfur dioxide	7
2.6 Γιατί μετά επιλέχθηκε η chlorides	8
2.7 Πώς το λύσαμε	8
2.8 Αποτελέσματα	9
2.9 Πώς προκύπτουν αυτές οι τιμές	9
2.10 Ερμηνεία	12
3 Ερώτημα 2: Bagging με 200 εκτιμητές	12

3.1	Τι ζητάει το ερώτημα	12
3.2	Η πιο απλή ιδέα του Bagging	13
3.3	Από πού βγαίνουν τα 200 διαφορετικά δέντρα	13
3.4	Άρα τι έκανε το Bagging στη δική μας εργασία	14
3.5	Θεωρητικό υπόβαθρο	14
3.6	Τι ακριβώς κάνει το Bagging στα δικά μας δεδομένα	15
3.7	Πώς γίνεται η τελική ψηφοφορία	15
3.8	Γιατί το Bagging βοηθά τα δέντρα	16
3.9	Πώς το λύσαμε	16
3.10	Αποτελέσματα	18
3.11	Πώς προκύπτουν οι αριθμοί του Bagging	18
3.12	Σύγκριση με το Ερώτημα 1	19
3.13	Γιατί το Bagging βελτιώθηκε τόσο σε σχέση με το απλό δέντρο	20
3.14	Σημαντικότητα χαρακτηριστικών	21
3.15	Συμπέρασμα Ερωτήματος 2	22
4	Ερώτημα 3: Random Forest με 200 εκτιμητές και $m = p/2$	22
4.1	Τι ζητάει το ερώτημα	22
4.2	Θεωρητικό υπόβαθρο	22
4.3	Γιατί δεν αρκεί πάντα το Bagging	23
4.4	Τα δύο επίπεδα τυχαιότητας στο Random Forest	24
4.4.1	Πρώτο επίπεδο: τυχαίες γραμμές του training set	24
4.4.2	Δεύτερο επίπεδο: τυχαίες μεταβλητές σε κάθε split	24
4.5	Τι σημαίνει $m = p/2$	25
4.6	Πώς το λύσαμε	25
4.7	Πώς παίρνει μία πρόβλεψη το Random Forest	26
4.8	Αποτελέσματα	27
4.9	Σύγκριση με τα Ερωτήματα 1 και 2	28
4.10	Σημαντικότητα χαρακτηριστικών	29
4.11	Συμπέρασμα Ερωτήματος 3	30
5	Ερώτημα 4: Boosting με 200 επιμέρους μοντέλα	30
5.1	Τι ζητάει το ερώτημα	30
5.2	Θεωρητικό υπόβαθρο	31
5.3	Η βασική διαφορά από Bagging και Random Forest	31
5.4	Τι σημαίνει ότι το μοντέλο διορθώνει τα λάθη	32
5.5	Τι είναι weak learner	33
5.6	Γιατί χρησιμοποιούμε $\max_depth=1$	33
5.7	Τι σημαίνει learning rate	33
5.8	Η σχέση ανάμεσα σε $n_estimators$ και $learning_rate$	34
5.9	Πώς παίρνει τελική απόφαση το Boosting	35
5.10	Πώς το λύσαμε	35
5.11	Αποτελέσματα	37
5.12	Confusion matrix	37
5.13	Σύγκριση με τα προηγούμενα μοντέλα	39
5.14	Σημαντικότητα χαρακτηριστικών	39
5.15	Συμπέρασμα Ερωτήματος 4	41
6	Ερώτημα 5: Test error του δέντρου συναρτήσει του βάθους	41

6.1	Τι ζητάει το ερώτημα	41
6.2	Θεωρητικό υπόβαθρο	41
6.3	Τι είναι test error	42
6.4	Πώς το λύσαμε	42
6.5	Αριθμητικά αποτελέσματα	43
6.6	Τι παρατηρούμε	44
6.7	Γιατί μετά το βάθος 6 χειροτερεύει	44
6.8	Συμπέρασμα Ερωτήματος 5	44
7	Συγκεντρωτική εικόνα μέχρι το Ερώτημα 5	45
A'	Αρχεία που χρησιμοποιούνται στο ενιαίο explanatory report	45

1 Γενικό πλαίσιο της εργασίας

Η 4η εργασία χρησιμοποιεί τα δεδομένα της 3ης εργασίας. Ο στόχος είναι να προβλέψουμε τον τύπο κρασιού, δηλαδή αν ένα κρασί είναι λευκό ή κόκκινο.

Η μεταβλητή που θέλουμε να προβλέψουμε είναι:

wine_type

και μπορεί να πάρει δύο τιμές:

Τιμή	Σημασία
red	κόκκινο κρασί
white	λευκό κρασί

Επειδή οι αλγόριθμοι μηχανικής μάθησης δουλεύουν πιο εύκολα με αριθμητικές τιμές, κωδικοποιήσαμε τη μεταβλητή στόχο ως:

red = 0 και white = 1.

1.1 Προβλεπτικές μεταβλητές

Η εκφώνηση ζητά να χρησιμοποιηθούν όλες οι διαθέσιμες μεταβλητές εκτός από τη quality. Η wine_type δεν μπαίνει φυσικά στους predictors, επειδή είναι η μεταβλητή που προσπαθούμε να προβλέψουμε.

Άρα οι προβλεπτικές μεταβλητές είναι 11:

fixed acidity	volatile acidity	citric acid
residual sugar	chlorides	free sulfur dioxide
total sulfur dioxide	density	pH
sulphates	alcohol	

1.2 Training set και test set

Τα δεδομένα χωρίστηκαν σε:

70% training set και 30% test set.

Το training set είναι το κομμάτι των δεδομένων από το οποίο μαθαίνει το μοντέλο. Το test set είναι το κομμάτι που κρατάμε στην άκρη και το χρησιμοποιούμε μόνο στο τέλος, για να δούμε αν το μοντέλο μπορεί να προβλέψει σωστά δεδομένα που δεν είχε δει κατά την εκπαίδευση.

Σύνολο	Πλήθος	Ποσοστό
Training set	3706	70%
Test set	1589	30%

Χρησιμοποιήθηκε:

random_state = 6931

ώστε ο διαχωρισμός να μπορεί να αναπαραχθεί ακριβώς. Επίσης χρησιμοποιήθηκε `stratify=y`, ώστε η αναλογία λευκών και κόκκινων κρασιών να παραμείνει παρόμοια στο training και στο test set.

1.3 Πώς μετράμε την απόδοση

Το βασικό μέτρο απόδοσης που ζητά η εργασία είναι η test accuracy:

$$\text{Test Accuracy} = \frac{\text{σωστές προβλέψεις στο test set}}{\text{σύνολο παρατηρήσεων στο test set}}.$$

Επίσης χρησιμοποιούμε το test error:

$$\text{Test Error} = 1 - \text{Test Accuracy}.$$

Αν ένα μοντέλο έχει accuracy 0.9660, αυτό σημαίνει ότι ταξινομεί σωστά περίπου το 96.60% των test παρατηρήσεων. Αντίστοιχα, test error 0.0340 σημαίνει ότι κάνει λάθος περίπου στο 3.40% των test παρατηρήσεων.

2 Ερώτημα 1: Δέντρο ταξινόμησης μέγιστου βάθους 2

2.1 Τι ζητάει το ερώτημα

Το πρώτο ερώτημα είναι:

«Να γίνει πρόβλεψη/εκτίμηση του τύπου κρασιού με δέντρο ταξινόμησης μέγιστου βάθους 2.»

Με απλά λόγια, θέλουμε να φτιάξουμε ένα μοντέλο που χρησιμοποιεί κανόνες τύπου «αν ισχύει αυτό, πήγαινε αριστερά, αλλιώς πήγαινε δεξιά» για να αποφασίσει αν ένα κρασί είναι λευκό ή κόκκινο.

Ο περιορισμός `max_depth=2` σημαίνει ότι το δέντρο πρέπει να παραμείνει πολύ απλό. Μπορεί να κάνει το πολύ δύο επίπεδα ερωτήσεων πριν καταλήξει σε τελική πρόβλεψη.

2.2 Θεωρητικό υπόβαθρο

Ένα δέντρο ταξινόμησης μοιάζει με διάγραμμα αποφάσεων. Σε κάθε κόμβο κάνει μία ερώτηση για κάποια μεταβλητή. Για παράδειγμα:

Είναι το `chlorides` μικρότερο ή ίσο από 0.05;

Ανάλογα με την απάντηση, η παρατήρηση ακολουθεί διαφορετικό κλαδί. Στο τέλος φτάνει σε ένα φύλλο, όπου το δέντρο δίνει την τελική πρόβλεψη: red ή white.

Το πλεονέκτημα των δέντρων είναι ότι είναι εύκολα ερμηνεύσιμα. Μπορούμε να δούμε τους κανόνες τους και να καταλάβουμε πώς πήραν μια απόφαση. Το μειονέκτημά τους είναι ότι, αν αφεθούν να μεγαλώσουν πάρα πολύ, μπορούν να υπερπροσαρμοστούν στα training δεδομένα. Για αυτό η εκφώνηση ζητά περιορισμό στο βάθος.

2.3 Πώς αποφασίζει το δέντρο ποιες μεταβλητές θα χρησιμοποιήσει

Είναι σημαντικό να ξεκαθαρίσουμε κάτι: εμείς δεν επιλέξαμε χειροκίνητα τις μεταβλητές `total sulfur dioxide` και `chlorides`. Δεν είπαμε δηλαδή στο μοντέλο «χρησιμοποίησε αυτές τις δύο μεταβλητές». Αντίθετα, δώσαμε στο δέντρο όλες τις 11 προβλεπτικές μεταβλητές και ο αλγόριθμος αποφάσισε μόνος του ποιες μεταβλητές και ποια όρια τιμών δίνουν τους καλύτερους διαχωρισμούς.

Ο αλγόριθμος που χρησιμοποιεί το `DecisionTreeClassifier` του `scikit-learn` είναι τύπου CART. Η βασική ιδέα είναι η εξής:

1. Σε κάθε κόμβο, το δέντρο εξετάζει όλες τις διαθέσιμες προβλεπτικές μεταβλητές.
2. Για κάθε μεταβλητή δοκιμάζει πολλά πιθανά όρια διαχωρισμού (thresholds).
3. Για κάθε πιθανό διαχωρισμό υπολογίζει πόσο «καθαρές» γίνονται οι δύο ομάδες που δημιουργούνται.
4. Επιλέγει τον διαχωρισμό που βελτιώνει περισσότερο την καθαρότητα των ομάδων.

Στο δικό μας μοντέλο χρησιμοποιείται το προεπιλεγμένο κριτήριο Gini impurity. Για δύο κλάσεις, δηλαδή `red` και `white`, η Gini impurity ενός κόμβου είναι:

$$Gini = 1 - p_{red}^2 - p_{white}^2,$$

όπου p_{red} είναι το ποσοστό των κόκκινων κρασιών στον κόμβο και p_{white} το ποσοστό των λευκών κρασιών στον ίδιο κόμβο.

Αν ένας κόμβος έχει μόνο κόκκινα ή μόνο λευκά κρασιά, τότε είναι πλήρως καθαρός και η Gini impurity είναι 0. Αν ένας κόμβος έχει περίπου μισά κόκκινα και μισά λευκά, τότε είναι πιο «μπερδεμένος» και η Gini impurity είναι μεγαλύτερη.

2.4 Πώς επιλέγεται το threshold μίας μεταβλητής

Οι μεταβλητές μας είναι συνεχείς αριθμητικές μεταβλητές. Για παράδειγμα, η `chlorides` μπορεί να πάρει τιμές όπως 0.047, 0.048, 0.069, 0.070 κ.λπ. Το δέντρο δεν ξέρει από πριν ποιο όριο είναι καλό. Το ψάχνει.

Για μία μεταβλητή, η διαδικασία είναι πρακτικά η εξής:

1. Ταξινομεί τις τιμές της μεταβλητής στο training set.
2. Δοκιμάζει υποψήφια thresholds ανάμεσα σε διαδοχικές τιμές.
3. Για κάθε threshold χωρίζει τις παρατηρήσεις σε δύο ομάδες:

$$X_j \leq \text{threshold} \quad \text{και} \quad X_j > \text{threshold}.$$

4. Υπολογίζει τη Gini impurity των δύο νέων ομάδων.
5. Κρατά το threshold που μειώνει περισσότερο τη συνολική Gini impurity.

Η μείωση της ακαθαρσίας υπολογίζεται ως:

$$\Delta Gini = Gini_{\text{parent}} - \left(\frac{n_L}{n} Gini_L + \frac{n_R}{n} Gini_R \right),$$

όπου n είναι το πλήθος παρατηρήσεων στον αρχικό κόμβο, n_L και n_R είναι τα πλήθη παρατηρήσεων στον αριστερό και δεξιό κόμβο, και $Gini_L$, $Gini_R$ είναι οι αντίστοιχες Gini impurities.

Αρα το δέντρο ψάχνει τον διαχωρισμό που έχει τη μεγαλύτερη τιμή $\Delta Gini$.

2.5 Γιατί επιλέχθηκε πρώτα το total sulfur dioxide

Στον αρχικό κόμβο, δηλαδή πριν γίνει οποιοσδήποτε διαχωρισμός, το δέντρο εξέτασε όλες τις μεταβλητές. Για κάθε μεταβλητή βρήκε το καλύτερο δυνατό threshold της και υπολόγισε πόσο μειώνεται η Gini impurity.

Οι καλύτεροι υποψήφιοι πρώτοι διαχωρισμοί ήταν:

Πίνακας 1: Καλύτερο πρώτο split ανά μεταβλητή, ταξινομημένο κατά μείωση Gini.

Μεταβλητή	Καλύτερο threshold	Μείωση Gini
total sulfur dioxide	67.5000	0.2066
chlorides	0.0615	0.2004
volatile acidity	0.4050	0.1139
free sulfur dioxide	19.5000	0.0813
sulphates	0.5250	0.0685
fixed acidity	8.5500	0.0633
residual sugar	4.1500	0.0576
density	0.9948	0.0539
citric acid	0.1250	0.0530
pH	3.2550	0.0274
alcohol	8.9500	0.0030

Βλέπουμε ότι η μεταβλητή total sulfur dioxide με threshold 67.50 δίνει τη μεγαλύτερη μείωση Gini, ίση με 0.2066. Για αυτό το δέντρο την επέλεξε ως πρώτο διαχωρισμό.

Η chlorides είναι πολύ κοντά, με μείωση Gini 0.2004, αλλά είναι λίγο χειρότερη από το total sulfur dioxide στον αρχικό κόμβο. Για αυτό δεν μπήκε πρώτη. Μπήκε όμως αμέσως μετά, στο δεύτερο επίπεδο του δέντρου.

2.6 Γιατί μετά επιλέχθηκε η chlorides

Αφού έγινε ο πρώτος διαχωρισμός με βάση τη total sulfur dioxide, τα training δεδομένα χωρίστηκαν σε δύο ομάδες:

- κρασιά με total sulfur dioxide ≤ 67.50 ,
- κρασιά με total sulfur dioxide > 67.50 .

Στη συνέχεια, επειδή το max_depth=2, το δέντρο μπορούσε να κάνει ακόμη έναν διαχωρισμό σε κάθε μία από αυτές τις δύο ομάδες. Σε κάθε ομάδα ξαναέκανε την ίδια διαδικασία: δοκίμασε όλες τις μεταβλητές και πολλά thresholds και επέλεξε τον καλύτερο διαχωρισμό.

Και στις δύο ομάδες, η καλύτερη μεταβλητή για το επόμενο split ήταν η chlorides. Τα πραγματικά splits του δέντρου ήταν:

Πίνακας 2: Τα splits που επέλεξε τελικά το δέντρο.

Κόμβος	Split	Πλήθος	Gini πριν	Μείωση Gini
Αρχικός κόμβος	total sulfur dioxide ≤ 67.50	3706	0.3713	0.2066
Αριστερός κόμβος	chlorides ≤ 0.0475	806	0.2464	0.1310
Δεξιός κόμβος	chlorides ≤ 0.0695	2900	0.1420	0.0477

Οι τιμές 0.0475 και 0.0695 εμφανίζονται συχνά στρογγυλοποιημένες ως 0.05 και 0.07 στους κανόνες του δέντρου. Δεν είναι τιμές που διαλέξαμε εμείς. Είναι όρια που προέκυψαν επειδή το δέντρο δοκίμασε πιθανούς διαχωρισμούς ανάμεσα στις διαδοχικές τιμές της μεταβλητής chlorides στο training set.

Ουσιαστικά, το δέντρο κατέληξε σε αυτές τις δύο μεταβλητές επειδή, στα training δεδομένα, αυτές έδιναν τους πιο καθαρούς διαχωρισμούς ανάμεσα σε κόκκινα και λευκά κρασιά. Αυτό δεν σημαίνει ότι οι άλλες μεταβλητές είναι άχρηστες. Σημαίνει ότι σε ένα πολύ μικρό δέντρο βάθους 2, οι πρώτοι και πιο αποτελεσματικοί διαχωρισμοί έγιναν με τη total sulfur dioxide και τη chlorides.

2.7 Πώς το λύσαμε

Ορίσαμε τους predictors και τη μεταβλητή στόχο:


```
features = [col for col in df.columns if col not in ("quality", "
    wine_type")]
X = df[features]
y = (df["wine_type"] == "white").astype(int)
```

Στη συνέχεια κάναμε το train/test split:

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=6931,
    stratify=y,
)
```

Τέλος εκπαιδεύσαμε το δέντρο:

```
model = DecisionTreeClassifier(max_depth=2, random_state=6931)
model.fit(X_train, y_train)
```

2.8 Αποτελέσματα

Πίνακας 3: Αποτελέσματα δέντρου ταξινόμησης με max_depth=2.

Μέτρο	Τιμή
Test accuracy	0.9339
Test error	0.0661
Σωστές προβλέψεις	1484
Λανθασμένες προβλέψεις	105

Ο confusion matrix ήταν:

Πίνακας 4: Confusion matrix για το Ερώτημα 1.

	Πρόβλεψη red	Πρόβλεψη white
Πραγματικό red	360	31
Πραγματικό white	74	1124

2.9 Πώς προκύπτουν αυτές οι τιμές

Οι αριθμοί του παραπάνω πίνακα δεν γράφτηκαν χειροκίνητα. Προέκυψαν αφού το εκπαιδευμένο δέντρο έκανε προβλέψεις στις 1589 παρατηρήσεις του test set. Στον κώδικα αυτό έγινε με τις εντολές:

```
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
```

Η μεταβλητή `y_test` περιέχει τις πραγματικές κατηγορίες των test παρατηρήσεων, ενώ η `y_pred` περιέχει τις προβλέψεις του μοντέλου. Ο confusion matrix συγκρίνει αυτά τα δύο:

πραγματική τιμή από `y_test`
σε σύγκριση με
πρόβλεψη από `y_pred`

Ας διαβάσουμε τώρα τα τέσσερα κελιά του confusion matrix ένα-ένα.

- **360**: κόκκινα κρασιά που ήταν πραγματικά κόκκινα και το μοντέλο τα προέβλεψε σωστά ως κόκκινα.
- **31**: κόκκινα κρασιά που ήταν πραγματικά κόκκινα, αλλά το μοντέλο τα προέβλεψε λάθος ως λευκά.
- **74**: λευκά κρασιά που ήταν πραγματικά λευκά, αλλά το μοντέλο τα προέβλεψε λάθος ως κόκκινα.
- **1124**: λευκά κρασιά που ήταν πραγματικά λευκά και το μοντέλο τα προέβλεψε σωστά ως λευκά.

Αν αθροίσουμε την πρώτη γραμμή, βρίσκουμε πόσα κόκκινα κρασιά υπήρχαν στο test set:

$$360 + 31 = 391.$$

Αν αθροίσουμε τη δεύτερη γραμμή, βρίσκουμε πόσα λευκά κρασιά υπήρχαν στο test set:

$$74 + 1124 = 1198.$$

Άρα το test set είχε συνολικά:

$$391 + 1198 = 1589$$

παρατηρήσεις, όπως περιμέναμε από τον διαχωρισμό 70%-30%.

Οι σωστές προβλέψεις είναι τα δύο κελιά της κύριας διαγωνίου του πίνακα:

$$\text{σωστές προβλέψεις} = 360 + 1124 = 1484.$$

Τα λάθη είναι τα δύο κελιά εκτός της κύριας διαγωνίου:

$$\text{λάθη} = 31 + 74 = 105.$$

Επομένως, η test accuracy προκύπτει ως:

$$\text{Test Accuracy} = \frac{\text{σωστές προβλέψεις}}{\text{σύνολο test παρατηρήσεων}} = \frac{1484}{1589} = 0.9339.$$

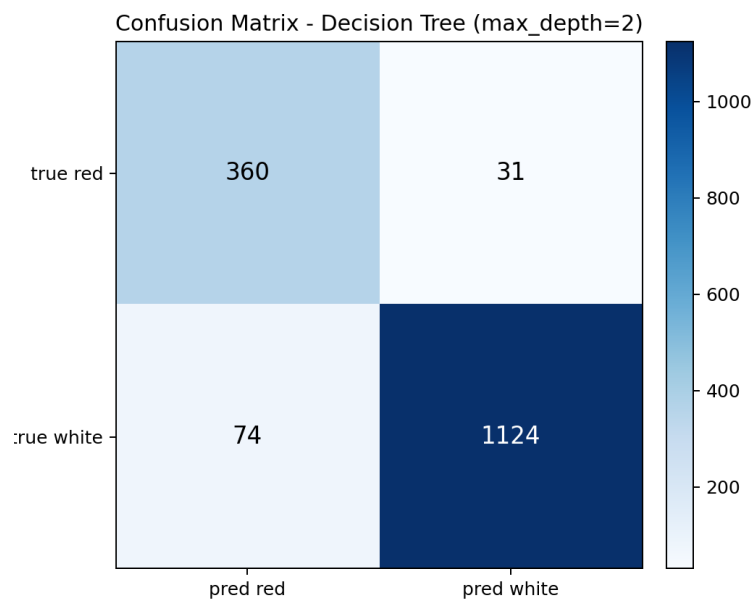
Το test error είναι το συμπλήρωμα της accuracy:

$$\text{Test Error} = 1 - 0.9339 = 0.0661.$$

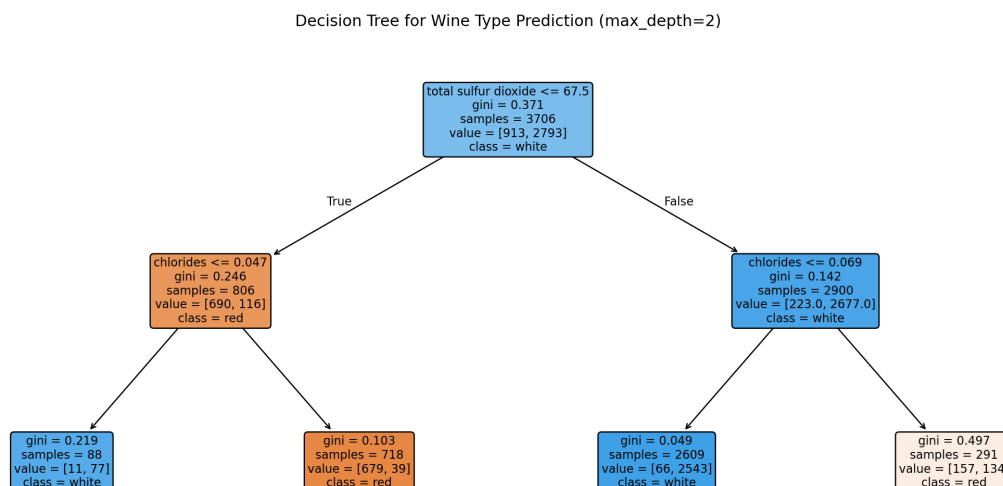
Ισοδύναμα, μπορούμε να το υπολογίσουμε απευθείας από τα λάθη:

$$\text{Test Error} = \frac{105}{1589} = 0.0661.$$

Άρα οι τιμές του πίνακα αποτελεσμάτων σημαίνουν ότι το δέντρο ταξινόμησης έκανε σωστή πρόβλεψη για 1484 από τα 1589 κρασιά του test set και λάθος πρόβλεψη για 105 κρασιά.



Σχήμα 1: Confusion matrix του δέντρου ταξινόμησης.



Σχήμα 2: Το δέντρο ταξινόμησης με μέγιστο βάθος 2.

2.10 Ερμηνεία

Το δέντρο βασίστηκε κυρίως στις μεταβλητές `total sulfur dioxide` και `chlorides`. Οι κανόνες του ήταν:

```
total sulfur dioxide <= 67.50
|-- chlorides <= 0.05 -> white
|-- chlorides > 0.05 -> red

total sulfur dioxide > 67.50
|-- chlorides <= 0.07 -> white
|-- chlorides > 0.07 -> red
```

Οι τιμές που εμφανίζονται στους κανόνες δεν επιλέχθηκαν από εμάς αυθαίρετα. Τις επέλεξε ο αλγόριθμος κατά την εκπαίδευση του δέντρου. Πιο συγκεκριμένα, τα πραγματικά `thresholds` ήταν 67.50 για τη `total sulfur dioxide`, 0.0475 για τη `chlorides` στον αριστερό κόμβο και 0.0695 για τη `chlorides` στον δεξιό κόμβο. Στο κείμενο των κανόνων οι δύο τελευταίες τιμές εμφανίζονται στρογγυλοποιημένες ως 0.05 και 0.07.

Το `DecisionTreeClassifier` δοκιμάζει πιθανούς διαχωρισμούς των μεταβλητών και επιλέγει εκείνους που κάνουν τις ομάδες όσο γίνεται πιο «καθαρές» ως προς την κλάση `red/white`. Με την προεπιλεγμένη ρύθμιση του `scikit-learn`, αυτό γίνεται με βάση το κριτήριο `Gini impurity`.

Επειδή έχουμε κωδικοποιήσει `red=0` και `white=1`, όταν στους κανόνες εμφανίζεται `class: 0` σημαίνει πρόβλεψη κόκκινου κρασιού, ενώ `class: 1` σημαίνει πρόβλεψη λευκού κρασιού.

Το μοντέλο πέτυχε `accuracy 0.9339`, δηλαδή ταξινόμησε σωστά περίπου το 93.39% των `test παρατηρήσεων`. Αυτό είναι αρκετά καλό αποτέλεσμα, ειδικά επειδή το δέντρο είναι πολύ απλό.

3 Ερώτημα 2: Bagging με 200 εκτιμητές

3.1 Τι ζητάει το ερώτημα

Το δεύτερο ερώτημα είναι:

«Να γίνει πρόβλεψη/εκτίμηση του τύπου κρασιού με τη μέθοδο Bagging, χρησιμοποιώντας 200 εκτιμητές.»

Εδώ δεν χρησιμοποιούμε ένα μόνο δέντρο. Χρησιμοποιούμε 200 δέντρα και συνδυάζουμε τις προβλέψεις τους. Αυτό είναι το βασικό σημείο του ερωτήματος: αντί να εμπιστευτούμε μία μόνο απόφαση, φτιάχνουμε πολλές διαφορετικές εκδοχές δέντρων και αφήνουμε την πλειοψηφία να αποφασίσει.

Πρακτικά, το ερώτημα μάς ζητά να απαντήσουμε στο εξής:

Αν το απλό δέντρο του Ερωτήματος 1 είχε κάποια λάθη, μπορούμε να βελτιώσουμε την πρόβλεψη χρησιμοποιώντας πολλά δέντρα αντί για ένα;

3.2 Η πιο απλή ιδέα του Bagging

Ας το πούμε όσο πιο απλά γίνεται.

Στο Ερώτημα 1 είχαμε **ένα** δέντρο. Δηλαδή ήταν σαν να ρωτάμε έναν μόνο «κριτή»:

Αυτό το κρασί είναι κόκκινο ή λευκό;

Ο ένας κριτής μπορεί να κάνει λάθος. Για παράδειγμα, μπορεί να μπερδευτεί σε κάποια κρασιά που μοιάζουν χημικά μεταξύ τους.

Στο Bagging δεν ρωτάμε έναν μόνο κριτή. Ρωτάμε **200 δέντρα**. Κάθε δέντρο δίνει τη δική του απάντηση και στο τέλος κρατάμε την απάντηση που ψήφισαν τα περισσότερα δέντρα.

Αρα, για ένα κρασί, μπορεί να γίνει κάτι τέτοιο:

Τι ψήφισαν τα δέντρα	Πλήθος ψήφων
white	160
red	40

Επειδή τα περισσότερα δέντρα ψήφισαν white, το Bagging θα προβλέψει white.

Αυτή είναι η κεντρική ιδέα:

Το Bagging είναι σαν να βάζουμε πολλά δέντρα να ψηφίσουν, αντί να αποφασίζει μόνο ένα δέντρο.

3.3 Από πού βγαίνουν τα 200 διαφορετικά δέντρα

Ίσως το πιο μπερδεμένο σημείο είναι αυτό: αφού έχουμε ένα training set, πώς φτιάχνουμε 200 διαφορετικά δέντρα;

Δεν χωρίζουμε τα δεδομένα 200 φορές σε training και test. Αυτό θα ήταν άλλο πράγμα. Το test set παραμένει το ίδιο και το χρησιμοποιούμε μόνο στο τέλος.

Αυτό που κάνει το Bagging είναι το εξής:

1. Παίρνει το training set.
2. Φτιάχνει από αυτό ένα «τυχαίο αντίγραφο» με δειγματοληψία με επανάθεση.
3. Εκπαιδεύει ένα δέντρο σε αυτό το τυχαίο αντίγραφο.
4. Επανακάνει την ίδια διαδικασία 200 φορές.

Το «με επανάθεση» σημαίνει ότι όταν διαλέγουμε μία γραμμή από το training set, μπορούμε να τη διαλέξουμε ξανά. Έτσι κάθε τυχαίο αντίγραφο είναι λίγο διαφορετικό από τα υπόλοιπα.

Ένα μικρό παράδειγμα:

Αρχικό training set: A, B, C, D, E

Ένα bootstrap δείγμα μπορεί να είναι:

A, C, C, E, B

Ένα άλλο bootstrap δείγμα μπορεί να είναι:

B, B, D, E, A

Βλέπουμε ότι τα δείγματα μοιάζουν, αλλά δεν είναι ακριβώς ίδια. Έτσι και τα δέντρα που εκπαιδεύονται πάνω τους δεν είναι ακριβώς ίδια.

3.4 Άρα τι έκανε το Bagging στη δική μας εργασία

Στη δική μας εργασία έγινε αυτό:

1. Χωρίσαμε τα δεδομένα μία φορά σε training και test.
2. Το training set είχε 3706 κρασιά.
3. Το Bagging έφτιαξε 200 τυχαία bootstrap δείγματα από αυτά τα 3706 training κρασιά.
4. Εκπαίδευσε 200 δέντρα, ένα για κάθε bootstrap δείγμα.
5. Μετά πήρε τα 1589 test κρασιά.
6. Για κάθε test κρασί, τα 200 δέντρα ψήφισαν red ή white.
7. Η τελική πρόβλεψη ήταν η πλειοψηφία των ψήφων.

Αν το πούμε σε μία πρόταση:

Το Bagging φτιάχνει πολλά λίγο διαφορετικά δέντρα και τα βάζει να ψηφίσουν.

3.5 Θεωρητικό υπόβαθρο

Το Bagging προέρχεται από τη φράση Bootstrap Aggregating. Η βασική ιδέα είναι:

1. Δημιουργούμε πολλά bootstrap δείγματα από το training set.
2. Εκπαιδεύουμε ένα δέντρο σε κάθε bootstrap δείγμα.
3. Κάθε δέντρο κάνει τη δική του πρόβλεψη.
4. Η τελική πρόβλεψη προκύπτει με πλειοψηφική ψηφοφορία.

Ένα bootstrap δείγμα δημιουργείται με δειγματοληψία **με επανάθεση**. Αυτό σημαίνει ότι μία παρατήρηση μπορεί να εμφανιστεί περισσότερες από μία φορές στο ίδιο bootstrap δείγμα, ενώ κάποια άλλη παρατήρηση μπορεί να μην εμφανιστεί καθόλου.

Για να γίνει πιο κατανοητό, ας φανταστούμε ένα μικρό training set με πέντε παρατηρήσεις:

A, B, C, D, E.

Ένα bootstrap δείγμα ίδιου μεγέθους θα μπορούσε να είναι:

A, C, C, E, B.

Εδώ η παρατήρηση C εμφανίστηκε δύο φορές, ενώ η D δεν εμφανίστηκε καθόλου. Αυτό δεν είναι λάθος. Είναι ακριβώς η ιδέα της δειγματοληψίας με επανάθεση.

3.6 Τι ακριβώς κάνει το Bagging στα δικά μας δεδομένα

Στα δικά μας δεδομένα, το training set έχει 3706 παρατηρήσεις. Όταν χρησιμοποιούμε Bagging με $n_estimators=200$, η διαδικασία είναι:

1. Δημιουργείται ένα bootstrap δείγμα από το training set.
2. Εκπαιδεύεται ένα δέντρο ταξινόμησης σε αυτό το δείγμα.
3. Η διαδικασία επαναλαμβάνεται 200 φορές.
4. Έτσι προκύπτουν 200 διαφορετικά δέντρα.
5. Για κάθε παρατήρηση του test set, τα 200 δέντρα ψηφίζουν αν είναι red ή white.
6. Η τελική πρόβλεψη είναι η κλάση με τις περισσότερες ψήφους.

Ένα σημαντικό σημείο είναι ότι **δεν κάνουμε 200 διαφορετικούς διαχωρισμούς σε training και test set**. Το training/test split γίνεται μία φορά, όπως και στα υπόλοιπα ερωτήματα. Τα 200 δείγματα δημιουργούνται μόνο μέσα από το training set.

3.7 Πώς γίνεται η τελική ψηφοφορία

Επειδή έχουμε πρόβλημα ταξινόμησης, το Bagging χρησιμοποιεί πλειοψηφική ψηφοφορία. Για κάθε κρασί του test set, κάθε ένα από τα 200 δέντρα δίνει μία πρόβλεψη.

Για παράδειγμα, για ένα συγκεκριμένο κρασί μπορεί να έχουμε:

Πρόβλεψη	Πλήθος δέντρων
white	173
red	27

Σε αυτή την περίπτωση, η τελική πρόβλεψη του Bagging είναι white, επειδή τα περισσότερα δέντρα ψήφισαν white.

Αντίστοιχα, αν τα περισσότερα δέντρα ψήφισαν red, η τελική πρόβλεψη θα ήταν red.

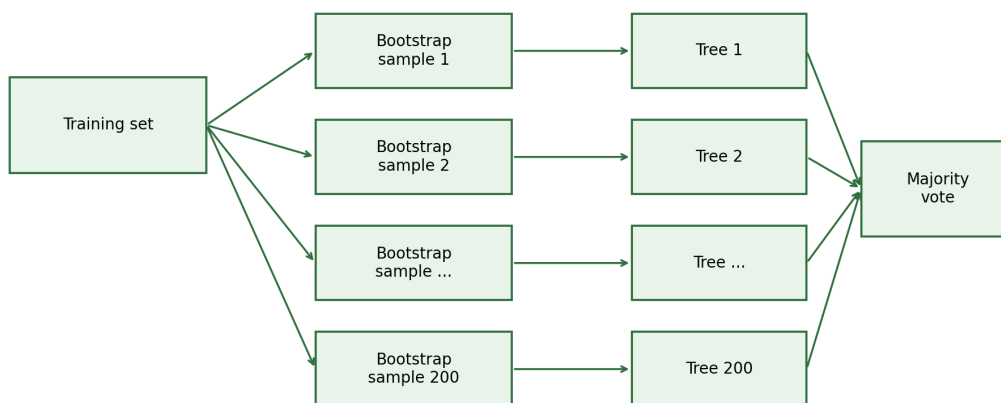
3.8 Γιατί το Bagging βοηθά τα δέντρα

Τα δέντρα ταξινόμησης είναι εύχρηστα και ερμηνεύσιμα, αλλά έχουν ένα γνωστό μειονέκτημα: μπορούν να είναι ασταθή. Μικρές αλλαγές στα training δεδομένα μπορεί να οδηγήσουν σε διαφορετικό δέντρο.

Το Bagging μειώνει αυτό το πρόβλημα επειδή δεν βασίζεται σε ένα μόνο δέντρο. Αντίθετα, δημιουργεί πολλά δέντρα σε διαφορετικά bootstrap δείγματα. Κάθε δέντρο μπορεί να έχει τα δικά του λάθη, αλλά όταν συνδυάζουμε 200 δέντρα, τα τυχαία λάθη τείνουν να εξομαλύνονται.

Με απλά λόγια:

Το Bagging δεν κάνει απαραίτητα κάθε δέντρο καλύτερο. Κάνει όμως την τελική απόφαση πιο σταθερή, επειδή βασίζεται στη συλλογική απόφαση πολλών δέντρων.



Bagging: many trees are trained on different bootstrap samples, then their votes are combined.

Σχήμα 3: Η βασική ιδέα του Bagging.

3.9 Πώς το λύσαμε

Χρησιμοποιήσαμε τον ίδιο διαχωρισμό training/test με το Ερώτημα 1, ώστε η σύγκριση να είναι δίκαιη. Αν αλλάζαμε το test set, δεν θα ξέραμε αν η διαφορά στην απόδοση οφείλεται στο μοντέλο ή απλώς σε διαφορετικά test δεδομένα.

Οι predictors και η μεταβλητή στόχος ορίστηκαν όπως πριν:

```
features = [col for col in df.columns if col not in ("quality", "
    wine_type")]
X = df[features]
y = (df["wine_type"] == "white").astype(int)
```

Στη συνέχεια έγινε το ίδιο train/test split:

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
```



```
y,  
test_size=0.30,  
random_state=6931,  
stratify=y,  
)
```

Το Bagging μοντέλο ορίστηκε ως:

```
model = BaggingClassifier(  
    estimator=DecisionTreeClassifier(random_state=6931),  
    n_estimators=200,  
    random_state=6931,  
    n_jobs=-1,  
)
```

Ας εξηγήσουμε αναλυτικά τις παραμέτρους:

- `BaggingClassifier`: δηλώνει ότι χρησιμοποιούμε Bagging για ταξινόμηση.
- `estimator=DecisionTreeClassifier(...)`: δηλώνει ότι το κάθε επιμέρους μοντέλο είναι δέντρο ταξινόμησης.
- `n_estimators=200`: δηλώνει ότι θα δημιουργηθούν και θα εκπαιδευτούν 200 δέντρα.
- `random_state=6931`: κάνει τη διαδικασία αναπαραγώγιμη.
- `n_jobs=-1`: επιτρέπει τη χρήση όλων των διαθέσιμων πυρήνων του υπολογιστή για ταχύτερη εκτέλεση.

Έπειτα εκπαιδεύσαμε το μοντέλο:

```
model.fit(X_train, y_train)
```

Με αυτή την εντολή, το Bagging δημιούργησε τα 200 bootstrap δείγματα και εκπαίδευσε τα 200 δέντρα.

Τέλος, κάναμε προβλέψεις στο test set:

```
y_pred = model.predict(X_test)  
accuracy = accuracy_score(y_test, y_pred)  
test_error = 1 - accuracy
```

Η μεταβλητή `y_pred` περιέχει την τελική πρόβλεψη του Bagging για κάθε test παρατήρηση, δηλαδή το αποτέλεσμα της πλειοψηφικής ψηφοφορίας των 200 δέντρων.

3.10 Αποτελέσματα

Πίνακας 5: Αποτελέσματα Bagging με 200 εκτιμητές.

Μέτρο	Τιμή
Test accuracy	0.9660
Test error	0.0340
Σωστές προβλέψεις	1535
Λανθασμένες προβλέψεις	54

Ο confusion matrix ήταν:

Πίνακας 6: Confusion matrix για το Bagging.

	Πρόβλεψη red	Πρόβλεψη white
Πραγματικό red	365	26
Πραγματικό white	28	1170

3.11 Πώς προκύπτουν οι αριθμοί του Bagging

Ο confusion matrix συγκρίνει την πραγματική κλάση κάθε test παρατήρησης με την πρόβλεψη του Bagging. Τα τέσσερα κελιά του πίνακα σημαίνουν:

- **365**: κόκκινα κρασιά που προβλέφθηκαν σωστά ως κόκκινα.
- **26**: κόκκινα κρασιά που προβλέφθηκαν λάθος ως λευκά.
- **28**: λευκά κρασιά που προβλέφθηκαν λάθος ως κόκκινα.
- **1170**: λευκά κρασιά που προβλέφθηκαν σωστά ως λευκά.

Άρα στο test set είχαμε:

$$365 + 26 = 391$$

κόκκινα κρασιά και:

$$28 + 1170 = 1198$$

λευκά κρασιά. Συνολικά:

$$391 + 1198 = 1589$$

test παρατηρήσεις.

Οι σωστές προβλέψεις είναι τα κελιά της κύριας διαγωνίου:

$$365 + 1170 = 1535.$$

Τα λάθη είναι τα δύο εκτός διαγωνίου κελιά:

$$26 + 28 = 54.$$

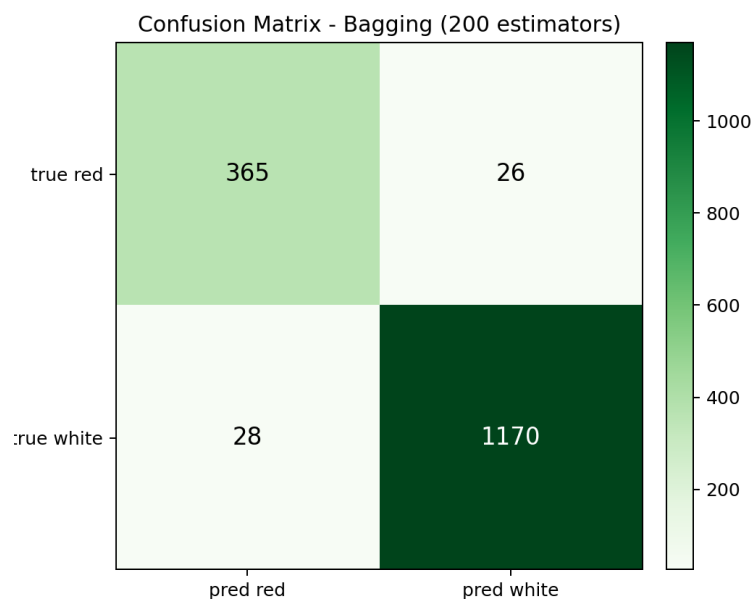
Άρα η test accuracy είναι:

$$\text{Test Accuracy} = \frac{1535}{1589} = 0.9660.$$

Και το test error είναι:

$$\text{Test Error} = \frac{54}{1589} = 0.0340.$$

Επομένως, το Bagging έκανε σωστή πρόβλεψη για 1535 από τα 1589 κρασιά του test set και λάθος πρόβλεψη για 54 κρασιά.



Σχήμα 4: Confusion matrix του Bagging.

3.12 Σύγκριση με το Ερώτημα 1

Πίνακας 7: Σύγκριση απλού δέντρου και Bagging.

Μοντέλο	Test Accuracy	Test Error
Decision Tree (max_depth=2)	0.9339	0.0661
Bagging (200 estimators)	0.9660	0.0340

Το Bagging έκανε 54 λάθη, ενώ το απλό δέντρο έκανε 105 λάθη. Άρα το Bagging έκανε:

$$105 - 54 = 51$$

λιγότερα λάθη.

Μπορούμε να δούμε τη βελτίωση και ως διαφορά στην accuracy:

$$0.9660 - 0.9339 = 0.0321.$$

Δηλαδή το Bagging αύξησε την ακρίβεια κατά περίπου 3.21 ποσοστιαίες μονάδες. Σε επίπεδο test error, η μείωση είναι:

$$0.0661 - 0.0340 = 0.0321.$$

Άρα το σφάλμα σχεδόν υποδιπλασιάστηκε. Το απλό δέντρο έκανε λάθος περίπου στο 6.61% των test παρατηρήσεων, ενώ το Bagging έκανε λάθος περίπου στο 3.40%.

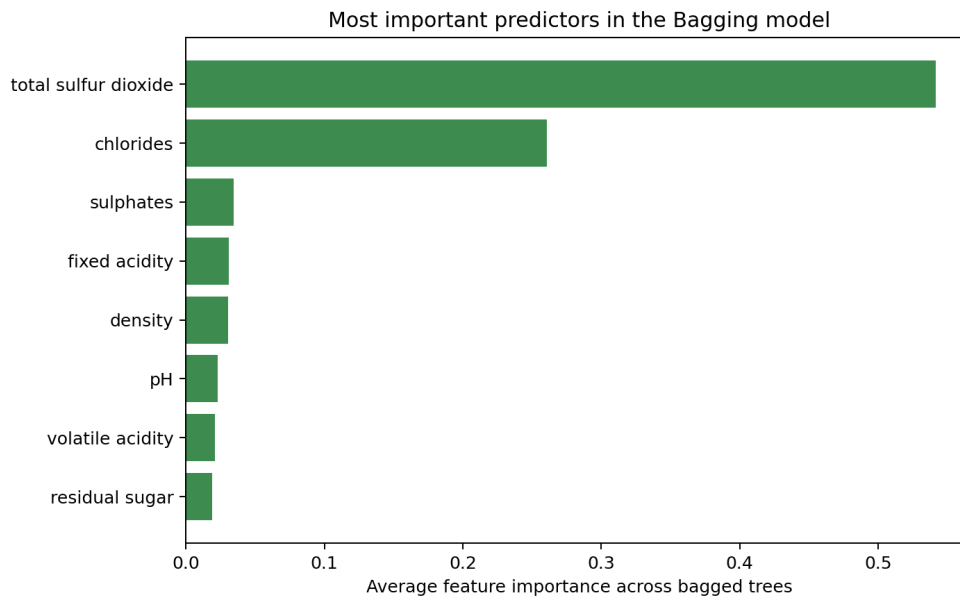
3.13 Γιατί το Bagging βελτιώθηκε τόσο σε σχέση με το απλό δέντρο

Το απλό δέντρο του Ερωτήματος 1 είχε `max_depth=2`, άρα ήταν πολύ περιορισμένο. Μπορούσε να χρησιμοποιήσει λίγους μόνο κανόνες. Αυτό το έκανε εύκολα ερμηνεύσιμο, αλλά ταυτόχρονα περιόριζε την προβλεπτική του δύναμη.

Το Bagging, αντίθετα, χρησιμοποίησε πολλά δέντρα. Κάθε δέντρο εκπαιδεύτηκε σε διαφορετικό bootstrap δείγμα και μπορούσε να αποτυπώσει διαφορετικές πλευρές των δεδομένων. Η τελική ψηφοφορία των 200 δέντρων ήταν πιο αξιόπιστη από την απόφαση ενός μόνο δέντρου.

Αυτό εξηγεί γιατί τα λάθη μειώθηκαν από 105 σε 54. Δεν σημαίνει ότι κάθε ένα από τα 200 δέντρα ήταν τέλειο. Σημαίνει ότι ο συνδυασμός τους έδωσε πιο σταθερό και ακριβές αποτέλεσμα.

3.14 Σημαντικότητα χαρακτηριστικών



Σχήμα 5: Οι σημαντικότερες μεταβλητές στο Bagging.

Οι σημαντικότερες μεταβλητές στο Bagging ήταν κυρίως οι total sulfur dioxide και chlorides. Αυτό συμφωνεί με το απλό δέντρο του Ερωτήματος 1.

Πίνακας 8: Μέση σημαντικότητα χαρακτηριστικών στο Bagging.

Μεταβλητή	Σημαντικότητα
total sulfur dioxide	0.5416
chlorides	0.2609
sulphates	0.0347
fixed acidity	0.0312
density	0.0303
pH	0.0228
volatile acidity	0.0211
residual sugar	0.0187
citric acid	0.0139
alcohol	0.0135
free sulfur dioxide	0.0112

Η σημαντικότητα χαρακτηριστικών δεν σημαίνει ότι οι υπόλοιπες μεταβλητές δεν χρησιμοποιήθηκαν ποτέ. Σημαίνει ότι, κατά μέσο όρο στα 200 δέντρα, οι μεταβλητές total sulfur dioxide και chlorides συνέβαλαν περισσότερο στους διαχωρισμούς που μείωναν την ακαθαρσία των κόμβων.

3.15 Συμπέρασμα Ερωτήματος 2

Το Bagging με 200 εκτιμητές πέτυχε test accuracy 0.9660. Η απόδοση είναι σημαντικά καλύτερη από το απλό δέντρο, επειδή το Bagging δεν βασίζεται σε ένα μόνο δέντρο αλλά σε ψηφοφορία πολλών δέντρων.

Το βασικό συμπέρασμα είναι ότι η χρήση πολλών bootstrap δειγμάτων και πολλών δέντρων μειώνει την αστάθεια ενός μεμονωμένου δέντρου. Έτσι, το μοντέλο κάνει λιγότερα λάθη στο test set και γενικεύει καλύτερα. Για αυτό το Bagging είναι μία από τις πιο κλασικές τεχνικές βελτίωσης των δέντρων απόφασης.

4 Ερώτημα 3: Random Forest με 200 εκτιμητές και $m = p/2$

4.1 Τι ζητάει το ερώτημα

Το τρίτο ερώτημα είναι:

«Να γίνει πρόβλεψη/εκτίμηση του τύπου κρασιού με Random Forest, χρησιμοποιώντας 200 εκτιμητές και τυχαίο πλήθος υποψήφιων χαρακτηριστικών $m = p/2$.»

Εδώ χρησιμοποιούμε ξανά πολλά δέντρα, όπως στο Bagging. Η διαφορά είναι ότι στο Random Forest κάθε split κάθε δέντρου δεν εξετάζει όλες τις μεταβλητές, αλλά μόνο ένα τυχαίο υποσύνολο.

Με πολύ απλά λόγια, το ερώτημα δεν ζητά να φτιάξουμε ένα μόνο δέντρο. Ζητά να φτιάξουμε ένα **δάσος** από 200 δέντρα αποφάσεων. Κάθε δέντρο δίνει τη δική του πρόβλεψη για το αν ένα κρασί είναι κόκκινο ή λευκό, και στο τέλος κρατάμε την απάντηση που ψήφισαν τα περισσότερα δέντρα.

Το επιπλέον σημείο που πρέπει να προσέξουμε είναι το $m = p/2$. Αυτό σημαίνει ότι, όταν ένα δέντρο προσπαθεί να αποφασίσει σε ποια μεταβλητή θα κάνει split, δεν κοιτάζει όλες τις διαθέσιμες μεταβλητές. Κοιτάζει μόνο ένα τυχαίο υποσύνολο μεταβλητών. Έτσι τα δέντρα δεν γίνονται όλα ίδια μεταξύ τους.

4.2 Θεωρητικό υπόβαθρο

Το Random Forest είναι σαν Bagging, αλλά με επιπλέον τυχαιότητα. Για να το καταλάβουμε σωστά, ας το δούμε σε σχέση με αυτά που κάναμε στο προηγούμενο ερώτημα.

Στο Bagging:

- παίρναμε πολλά bootstrap δείγματα από το training set,
- εκπαιδεύαμε ένα δέντρο σε κάθε bootstrap δείγμα,
- συνδυάζαμε τις προβλέψεις των δέντρων με ψηφοφορία.

Στο Random Forest κάνουμε τα ίδια, αλλά προσθέτουμε μία ακόμα ιδέα:

- κάθε δέντρο εξακολουθεί να εκπαιδεύεται σε bootstrap δείγμα,
- όμως σε κάθε split εξετάζεται μόνο τυχαίο υποσύνολο χαρακτηριστικών,
- από αυτό το μικρότερο υποσύνολο επιλέγεται το καλύτερο split,
- στο τέλος όλα τα δέντρα ψηφίζουν για την τελική κλάση.

Αυτό βοηθά γιατί κάνει τα δέντρα πιο διαφορετικά μεταξύ τους. Αν όλα τα δέντρα μοιάζουν πολύ, τότε μπορεί να κάνουν παρόμοια λάθη. Αν όμως είναι πιο διαφορετικά, η πλειοψηφική ψηφοφορία γίνεται πιο χρήσιμη.

4.3 Γιατί δεν αρκεί πάντα το Bagging

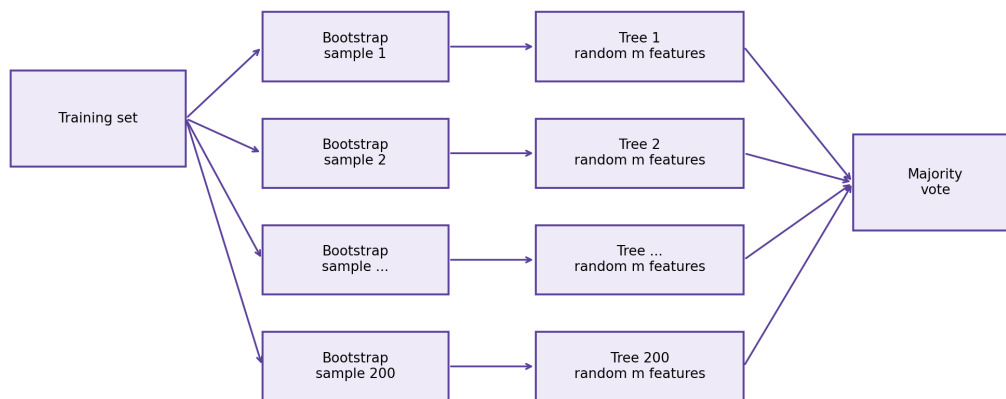
Στο Bagging, κάθε δέντρο μπορεί να εξετάσει όλες τις μεταβλητές σε κάθε split. Αν υπάρχει μία πολύ δυνατή μεταβλητή, όπως για παράδειγμα η `total sulfur dioxide`, πολλά δέντρα είναι πιθανό να τη χρησιμοποιήσουν πολύ νωρίς. Αυτό κάνει τα δέντρα καλύτερα μεμονωμένα, αλλά μπορεί να τα κάνει και αρκετά παρόμοια μεταξύ τους.

Το πρόβλημα είναι το εξής: αν τα 200 δέντρα μοιάζουν πολύ μεταξύ τους, τότε τα 200 δέντρα δεν μας δίνουν 200 πραγματικά διαφορετικές απόψεις. Είναι σαν να ρωτάμε πολλούς ανθρώπους που σκέφτονται σχεδόν με τον ίδιο τρόπο.

Το Random Forest προσπαθεί να μειώσει αυτό το πρόβλημα. Σε κάθε split λέει στο δέντρο:

«Μην κοιτάξεις όλες τις μεταβλητές. Θα σου δώσω τυχαία μόνο μερικές από αυτές. Διάλεξε το καλύτερο split μόνο ανάμεσα σε αυτές.»

Έτσι, αν σε κάποιο split δεν τύχει να είναι διαθέσιμη η πιο δυνατή μεταβλητή, το δέντρο αναγκάζεται να βρει καλή πληροφορία από κάποια άλλη μεταβλητή. Αυτό δημιουργεί πιο διαφορετικά δέντρα. Και όταν τα δέντρα είναι πιο διαφορετικά, η ψηφοφορία τους μπορεί να γενικεύει καλύτερα σε άγνωστα δεδομένα.



Random Forest = Bagging of trees + random subset of candidate features at each split ($m = p/2$).

Σχήμα 6: Η βασική ιδέα του Random Forest.

4.4 Τα δύο επίπεδα τυχειότητας στο Random Forest

Στο συγκεκριμένο μοντέλο υπάρχουν δύο διαφορετικά είδη τυχειότητας.

4.4.1 Πρώτο επίπεδο: τυχαίες γραμμές του training set

Όπως και στο Bagging, κάθε δέντρο δεν εκπαιδεύεται ακριβώς στο ίδιο training set. Για κάθε δέντρο φτιάχνεται ένα bootstrap δείγμα. Δηλαδή επιλέγονται τυχαία **ολόκληρες γραμμές** από το training set, με επανάθεση.

Αυτό σημαίνει ότι μία γραμμή μπορεί να εμφανιστεί περισσότερες από μία φορές στο bootstrap δείγμα, ενώ κάποια άλλη γραμμή μπορεί να μη χρησιμοποιηθεί καθόλου από το συγκεκριμένο δέντρο.

Δεν παίρνουμε τυχαίες τιμές από κάθε στήλη ξεχωριστά. Παίρνουμε ολόκληρες παρατηρήσεις κρασιών, ώστε να μη χαλάσει η σχέση που έχουν οι μεταβλητές μεταξύ τους.

4.4.2 Δεύτερο επίπεδο: τυχαίες μεταβλητές σε κάθε split

Το δεύτερο επίπεδο τυχειότητας είναι αυτό που κάνει το Random Forest να διαφέρει από το απλό Bagging. Όταν ένα δέντρο βρίσκεται σε έναν κόμβο και θέλει να αποφασίσει το επόμενο split, δεν επιτρέπεται να εξετάσει και τις 11 μεταβλητές. Εξετάζει μόνο $m = 5$ τυχαία επιλεγμένες μεταβλητές.

Για παράδειγμα, σε έναν κόμβο μπορεί οι 5 υποψήφιες μεταβλητές να είναι:

chlorides, density, pH, alcohol, sulphates.

Σε έναν άλλο κόμβο, ακόμα και μέσα στο ίδιο δέντρο, μπορεί οι 5 υποψήφιες μεταβλητές να είναι διαφορετικές:

total sulfur dioxide, residual sugar, citric acid, free sulfur dioxide, fixed acidity.

Αυτό γίνεται ξανά και ξανά, σε κάθε split και σε κάθε δέντρο.

4.5 Τι σημαίνει $m = p/2$

Στο πρόβλημά μας οι προβλεπτικές μεταβλητές είναι:

$$p = 11.$$

Οι 11 αυτές μεταβλητές είναι όλες οι μεταβλητές του dataset εκτός από:

- τη `wine_type`, γιατί αυτή είναι η μεταβλητή που θέλουμε να προβλέψουμε,
- την `quality`, γιατί η εκφώνηση λέει να μη χρησιμοποιηθεί.

Άρα οι `predictors` που χρησιμοποιούνται είναι:

<code>fixed acidity</code>	<code>volatile acidity</code>	<code>citric acid</code>
<code>residual sugar</code>	<code>chlorides</code>	<code>free sulfur dioxide</code>
<code>total sulfur dioxide</code>	<code>density</code>	<code>pH</code>
<code>sulphates</code>	<code>alcohol</code>	

Η εκφώνηση ζητά:

$$m = \frac{p}{2} = \frac{11}{2} = 5.5.$$

Επειδή το `scikit-learn` χρειάζεται ακέραιο πλήθος χαρακτηριστικών όταν δίνουμε το `max_features` ως αριθμό, χρησιμοποιήθηκε:

$$\text{max_features} = 5.$$

Άρα, σε κάθε split, το Random Forest εξέταζε τυχαία 5 από τις 11 μεταβλητές.

Το σημαντικό είναι ότι το $m = 5$ δεν σημαίνει ότι το μοντέλο χρησιμοποιεί μόνο 5 μεταβλητές συνολικά. Σημαίνει ότι **σε κάθε split** εξετάζει 5 τυχαίες μεταβλητές. Σε άλλο split μπορεί να εξετάσει άλλες 5. Επομένως, συνολικά το Random Forest μπορεί να αξιοποιήσει και τις 11 μεταβλητές, απλώς δεν τις εξετάζει όλες ταυτόχρονα σε κάθε απόφαση.

4.6 Πώς το λύσαμε

Ο υπολογισμός του m έγινε ως εξής:

```
features = [col for col in df.columns if col not in ("quality", "
    wine_type")]
p = len(features)
max_features = p // 2
```

Το μοντέλο ορίστηκε ως:

```
model = RandomForestClassifier(
    n_estimators=200,
    max_features=5,
    random_state=6931,
    n_jobs=-1,
)
```

Η διαδικασία εκπαίδευσης μπορεί να περιγραφεί βήμα-βήμα ως εξής:

1. Κρατήσαμε ως predictors όλες τις μεταβλητές εκτός από quality και wine_type.
2. Χρησιμοποιήσαμε το ίδιο training set και test set με τα προηγούμενα ερωτήματα.
3. Υπολογίσαμε ότι το πλήθος των predictors είναι $p = 11$.
4. Θέσαμε $m = p/2$, δηλαδή στην πράξη max_features=5.
5. Εκπαιδεύσαμε 200 δέντρα αποφάσεων.
6. Κάθε δέντρο εκπαιδεύτηκε σε διαφορετικό bootstrap δείγμα του training set.
7. Σε κάθε split κάθε δέντρου, εξετάστηκαν μόνο 5 τυχαίες μεταβλητές.
8. Για κάθε παρατήρηση του test set, τα 200 δέντρα έδωσαν 200 προβλέψεις.
9. Η τελική πρόβλεψη ήταν η κλάση με τις περισσότερες ψήφους.

Αν, για παράδειγμα, για ένα κρασί τα 200 δέντρα ψήφισαν ως εξής:

184 δέντρα λένε white και 16 δέντρα λένε red,

τότε το Random Forest θα προέβλεπε ότι το κρασί είναι white.

4.7 Πώς παίρνει μία πρόβλεψη το Random Forest

Για να γίνει πιο καθαρό, ας πάρουμε νοητά μία παρατήρηση από το test set, δηλαδή ένα κρασί που το μοντέλο δεν έχει δει στην εκπαίδευση. Η παρατήρηση αυτή έχει τιμές για τις 11 προβλεπτικές μεταβλητές.

Η παρατήρηση περνά από το πρώτο δέντρο και παίρνει μία πρόβλεψη. Μετά περνά από το δεύτερο δέντρο και παίρνει άλλη μία πρόβλεψη. Αυτό συνεχίζεται μέχρι και το δέντρο 200.

Στο τέλος μπορεί να έχουμε κάτι σαν:

	Ψήφοι για red	Ψήφοι για white
Παράδειγμα test κρασιού	23	177

Επειδή οι περισσότερες ψήφοι είναι για white, η τελική πρόβλεψη είναι white. Αυτός είναι ο λόγος που λέμε ότι το Random Forest είναι ensemble μέθοδος: δεν εμπιστεύεται ένα μόνο μοντέλο, αλλά συνδυάζει πολλά μοντέλα.

4.8 Αποτελέσματα

Πίνακας 9: Αποτελέσματα Random Forest.

Μέτρο	Τιμή
Πλήθος predictors p	11
$p/2$	5.5
Χρησιμοποιημένο max_features	5
Test accuracy	0.9666
Test error	0.0334
Σωστές προβλέψεις	1536
Λανθασμένες προβλέψεις	53

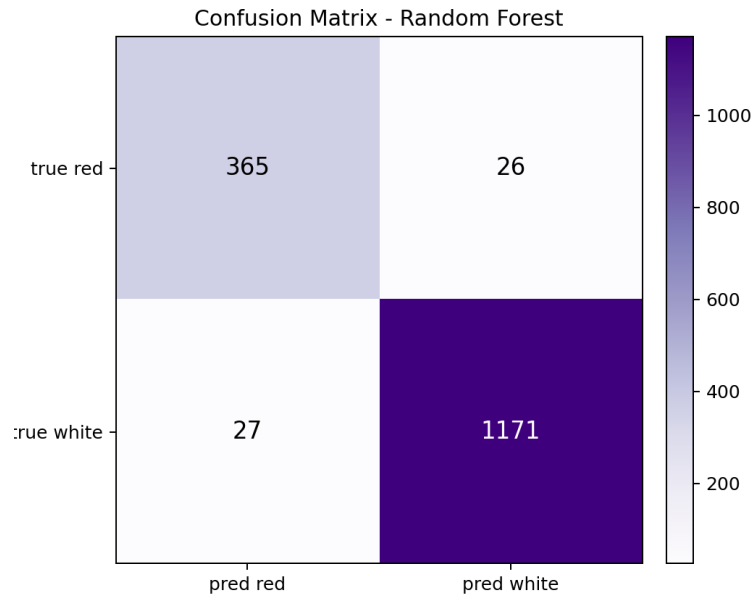
Ας διαβάσουμε τον πίνακα με απλά λόγια:

- η ακρίβεια ήταν 0.9666, δηλαδή περίπου 96.66%,
- το test error ήταν 0.0334, δηλαδή περίπου 3.34%,
- από τις 1589 παρατηρήσεις του test set, οι 1536 ταξινομήθηκαν σωστά,
- οι 53 ταξινομήθηκαν λάθος.

Ο confusion matrix ήταν:

Πίνακας 10: Confusion matrix για το Random Forest.

	Πρόβλεψη red	Πρόβλεψη white
Πραγματικό red	365	26
Πραγματικό white	27	1171



Σχήμα 7: Confusion matrix του Random Forest.

Ο confusion matrix μάς λέει πιο συγκεκριμένα πού έγιναν τα λάθη:

- 365 κόκκινα κρασιά αναγνωρίστηκαν σωστά ως κόκκινα,
- 26 κόκκινα κρασιά προβλέφθηκαν λανθασμένα ως λευκά,
- 27 λευκά κρασιά προβλέφθηκαν λανθασμένα ως κόκκινα,
- 1171 λευκά κρασιά αναγνωρίστηκαν σωστά ως λευκά.

Άρα το μοντέλο κάνει λάθη και στις δύο κατευθύνσεις, αλλά τα λάθη είναι λίγα σε σχέση με το συνολικό μέγεθος του test set.

4.9 Σύγκριση με τα Ερωτήματα 1 και 2

Πίνακας 11: Σύγκριση των τριών πρώτων μοντέλων.

Μοντέλο	Test Accuracy	Test Error
Decision Tree (max_depth=2)	0.9339	0.0661
Bagging (200 estimators)	0.9660	0.0340
Random Forest (200 estimators, $m = 5$)	0.9666	0.0334

Το Random Forest έκανε 53 λάθη, δηλαδή ένα λιγότερο από το Bagging και 52 λιγότερα από το απλό δέντρο.

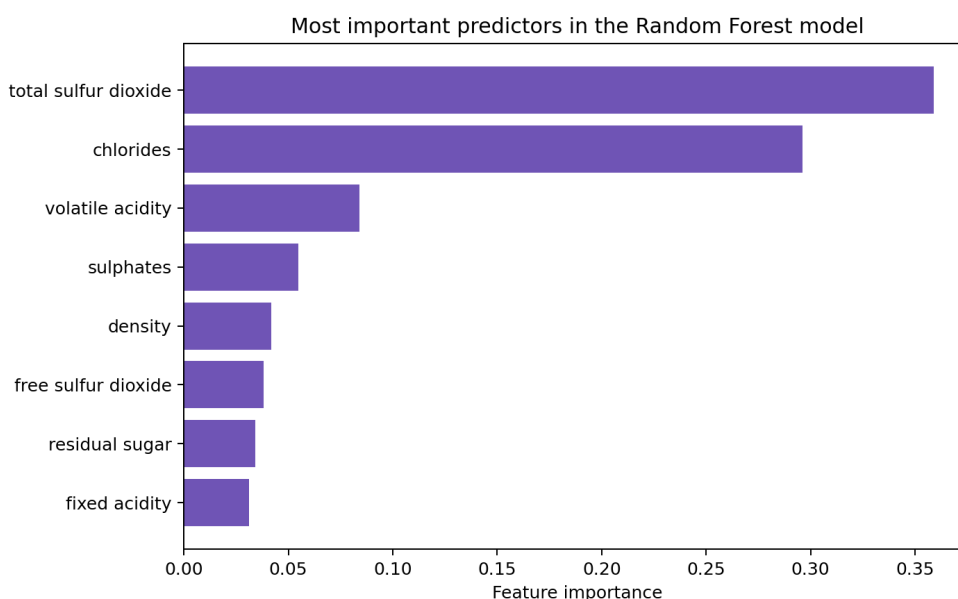
Η βελτίωση σε σχέση με το απλό δέντρο είναι μεγάλη. Αυτό είναι αναμενόμενο, γιατί το απλό δέντρο του Ερωτήματος 1 είχε μέγιστο βάθος 2 και άρα ήταν πολύ περιορισμένο. Μπορούσε να κάνει μόνο λίγα splits.

Η βελτίωση σε σχέση με το Bagging είναι πολύ μικρή. Αυτό δεν σημαίνει ότι το Random Forest απέτυχε. Σημαίνει ότι, για το συγκεκριμένο dataset,

το Bagging ήταν ήδη πολύ δυνατό. Οι μεταβλητές total sulfur dioxide και chlorides διαχωρίζουν αρκετά καλά τα κόκκινα από τα λευκά κρασιά, άρα ακόμα και το Bagging έχει ήδη υψηλή ακρίβεια.

Το Random Forest κέρδισε οριακά επειδή μείωσε λίγο περισσότερο την ομοιότητα ανάμεσα στα δέντρα. Όταν τα δέντρα είναι λιγότερο συσχετισμένα μεταξύ τους, τα λάθη τους τείνουν να ακυρώνονται καλύτερα μέσω της ψηφοφορίας.

4.10 Σημαντικότητα χαρακτηριστικών



Σχήμα 8: Οι σημαντικότερες μεταβλητές στο Random Forest.

Οι σημαντικότερες μεταβλητές ήταν και πάλι οι total sulfur dioxide και chlorides. Αυτό είναι συνεπές με όσα είδαμε και στα δύο προηγούμενα ερωτήματα.

Η σημαντικότητα χαρακτηριστικών στο Random Forest υπολογίζεται με βάση το πόσο βοηθά κάθε μεταβλητή στη μείωση της ακαθαρσίας των κόμβων μέσα στα δέντρα. Με πιο απλά λόγια, μία μεταβλητή θεωρείται σημαντική όταν χρησιμοποιείται συχνά σε splits που κάνουν τους κόμβους πιο καθαρούς, δηλαδή όταν βοηθά να ξεχωρίζουν καλύτερα τα κόκκινα από τα λευκά κρασιά.

Πρέπει όμως να το ερμηνεύουμε προσεκτικά. Η σημαντικότητα δεν σημαίνει ότι η μεταβλητή είναι η αιτία που ένα κρασί είναι κόκκινο ή λευκό. Σημαίνει ότι, μέσα στο συγκεκριμένο dataset και με το συγκεκριμένο μοντέλο, η μεταβλητή ήταν χρήσιμη για την ταξινόμηση.

4.11 Συμπέρασμα Ερωτήματος 3

Το Random Forest με 200 δέντρα και `max_features=5` πέτυχε test accuracy 0.9666. Η απόδοση ήταν πολύ καλύτερη από το απλό δέντρο και οριακά καλύτερη από το Bagging. Η βελτίωση σε σχέση με το Bagging είναι μικρή, αλλά το Random Forest παραμένει θεωρητικά πιο ισχυρή παραλλαγή, επειδή μειώνει τη συσχέτιση μεταξύ των δέντρων μέσω της τυχαίας επιλογής χαρακτηριστικών.

Αρα, η βασική ιδέα που πρέπει να κρατήσουμε είναι η εξής: το Random Forest βελτιώνει το Bagging όχι επειδή φτιάχνει απλώς περισσότερα δέντρα, αλλά επειδή κάνει τα δέντρα πιο διαφορετικά μεταξύ τους. Αυτό το πετυχαίνει επιτρέποντας σε κάθε split να δει μόνο ένα τυχαίο μέρος των διαθέσιμων μεταβλητών.

5 Ερώτημα 4: Boosting με 200 επιμέρους μοντέλα

5.1 Τι ζητάει το ερώτημα

Το τέταρτο ερώτημα είναι:

«Να γίνει πρόβλεψη/εκτίμηση του τύπου κρασιού με τη μέθοδο Boosting, χρησιμοποιώντας 200 επιμέρους μοντέλα, συντελεστή μάθησης 0.1 και μέγιστο βάθος 1.»

Με απλά λόγια, θέλουμε ξανά να προβλέψουμε αν ένα κρασί είναι λευκό ή κόκκινο, αλλά αυτή τη φορά με τη μέθοδο **Boosting**. Η βασική ιδέα του Boosting είναι διαφορετική από το Bagging και το Random Forest. Στο Bagging και στο Random Forest τα δέντρα μπορούν να εκπαιδευτούν σχετικά ανεξάρτητα μεταξύ τους. Στο Boosting, όμως, τα μοντέλα χτίζονται **διαδοχικά**.

Αυτό είναι το πιο σημαντικό σημείο του ερωτήματος. Δεν φτιάχνουμε 200 δέντρα που δουλεύουν ανεξάρτητα και μετά απλώς ψηφίζουν. Φτιάχνουμε 200 μικρά δέντρα με σειρά. Το δεύτερο δέντρο εκπαιδεύεται αφού έχει ήδη φτιαχτεί το πρώτο, το τρίτο αφού έχουν ήδη φτιαχτεί τα δύο πρώτα, και ούτω καθεξής. Κάθε νέο δέντρο προσπαθεί να προσθέσει μία μικρή διόρθωση στο συνολικό μοντέλο.

Δηλαδή:

- πρώτα εκπαιδεύεται ένα απλό μοντέλο,
- μετά εκπαιδεύεται ένα δεύτερο μοντέλο που προσπαθεί να διορθώσει τα λάθη του πρώτου,
- μετά ένα τρίτο που προσπαθεί να βελτιώσει ακόμη περισσότερο το σύνολο,
- και η διαδικασία συνεχίζεται μέχρι να φτάσουμε στα 200 μοντέλα.

Στο ερώτημα ζητούνται συγκεκριμένες τιμές:

`n_estimators = 200,      learning_rate = 0.1,      max_depth = 1.`

5.2 Θεωρητικό υπόβαθρο

Το Boosting είναι μία ensemble μέθοδος. Αυτό σημαίνει ότι δεν βασίζεται σε ένα μόνο μοντέλο, αλλά συνδυάζει πολλά μικρότερα μοντέλα.

Η σημαντική διαφορά είναι ότι στο Boosting τα μοντέλα δεν φτιάχνονται όλα ανεξάρτητα. Φτιάχνονται το ένα μετά το άλλο. Κάθε νέο μοντέλο προσπαθεί να βελτιώσει το συνολικό μοντέλο που έχει δημιουργηθεί μέχρι εκείνη τη στιγμή.

Μπορούμε να το σκεφτούμε σαν μία επαναληπτική διαδικασία διόρθωσης:

1. Το πρώτο μικρό δέντρο κάνει μία αρχική προσπάθεια ταξινόμησης.
2. Το μοντέλο βλέπει σε ποιες παρατηρήσεις τα πήγε καλά και σε ποιες όχι.
3. Το επόμενο μικρό δέντρο προσπαθεί να βοηθήσει κυρίως εκεί όπου το υπάρχον μοντέλο δυσκολεύτηκε.
4. Η νέα γνώση προστίθεται στο συνολικό μοντέλο, αλλά με μικρό βάρος, επειδή έχουμε `learning_rate=0.1`.
5. Η διαδικασία επαναλαμβάνεται 200 φορές.

Αρα το Boosting δεν λειτουργεί με τη λογική «πολλά ανεξάρτητα μοντέλα και ψηφοφορία», όπως το Bagging και το Random Forest. Λειτουργεί με τη λογική «χτίζω σταδιακά ένα δυνατό μοντέλο, προσθέτοντας κάθε φορά μία μικρή διόρθωση».

5.3 Η βασική διαφορά από Bagging και Random Forest

Επειδή έχουμε ήδη δει Bagging και Random Forest, αξίζει να ξεκαθαρίσουμε τη διαφορά τους από το Boosting.

Πίνακας 12: Διαισθητική σύγκριση Bagging, Random Forest και Boosting.

Μέθοδος	Πώς φτιάχνονται τα δέντρα	Κύρια ιδέα
Bagging	Πολλά δέντρα φτιάχνονται σε διαφορετικά bootstrap δείγματα.	Μειώνουμε την αστάθεια ενός δέντρου παίρνοντας μέσο όρο ή ψηφοφορία.
Random Forest	Όπως το Bagging, αλλά σε κάθε split εξετάζεται τυχαίο υποσύνολο μεταβλητών.	Κάνουμε τα δέντρα πιο διαφορετικά μεταξύ τους.
Boosting	Τα δέντρα φτιάχνονται διαδοχικά, το ένα μετά το άλλο.	Κάθε νέο δέντρο προσπαθεί να διορθώσει τις αδυναμίες του προηγούμενου συνόλου.

Επομένως, το Boosting είναι πιο «καθοδηγούμενο». Δεν αφήνει απλώς τα δέντρα να εκπαιδευτούν ανεξάρτητα. Κάθε νέο δέντρο εξαρτάται από το τι έχει ήδη μάθει το μοντέλο μέχρι εκείνη τη στιγμή.

5.4 Τι σημαίνει ότι το μοντέλο διορθώνει τα λάθη

Η φράση «διορθώνει τα λάθη» είναι χρήσιμη, αλλά θέλει λίγη προσοχή. Δεν σημαίνει ότι το δεύτερο δέντρο απλώς βρίσκει τις παρατηρήσεις που το πρώτο δέντρο ταξινομήσε λάθος και τις απομνημονεύει.

Σημαίνει ότι το συνολικό μοντέλο κοιτάζει πού οι προβλέψεις του δεν είναι αρκετά καλές και το επόμενο δέντρο προσπαθεί να προσθέσει πληροφορία προς τη σωστή κατεύθυνση.

Στην ταξινόμηση κρασιού, το μοντέλο προσπαθεί να μάθει έναν τρόπο να δίνει υψηλή πιθανότητα στην κλάση *white* όταν το κρασί είναι λευκό και υψηλή πιθανότητα στην κλάση *red* όταν το κρασί είναι κόκκινο. Αν για κάποιο λευκό κρασί το υπάρχον μοντέλο δεν είναι αρκετά σίγουρο ή γέρνει προς τη λάθος κλάση, τα επόμενα δέντρα προσπαθούν να μετακινήσουν την πρόβλεψη προς τη σωστή πλευρά.

Στο `GradientBoostingClassifier` αυτή η διόρθωση γίνεται με βάση την ιδέα του **gradient**. Δεν χρειάζεται για την εργασία να μπούμε σε βαθιά μαθηματικά, αλλά η διαίσθηση είναι η εξής:

Το μοντέλο κοιτάζει προς ποια κατεύθυνση πρέπει να αλλάξει τις προβλέψεις του ώστε να μειωθεί το λάθος, και κάθε νέο μικρό δέντρο κάνει ένα μικρό βήμα προς αυτή την κατεύθυνση.

Γι' αυτό η μέθοδος λέγεται Gradient Boosting.

5.5 Τι είναι weak learner

Στο Boosting συχνά χρησιμοποιούμε πολύ απλά μοντέλα, τα οποία λέγονται **weak learners**. Ένας weak learner δεν είναι απαραίτητα πολύ ισχυρός μόνος του. Μπορεί να κάνει αρκετά λάθη. Όμως, όταν συνδυάσουμε πολλούς weak learners σωστά, μπορούμε να δημιουργήσουμε ένα πολύ ισχυρό μοντέλο.

Στο δικό μας ερώτημα, κάθε επιμέρους μοντέλο είναι δέντρο με:

$$\text{max_depth} = 1.$$

Ένα δέντρο βάθους 1 λέγεται συχνά decision stump. Είναι ένα πολύ απλό δέντρο, το οποίο κάνει μόνο έναν βασικό διαχωρισμό. Από μόνο του δεν έχει μεγάλη εκφραστική δύναμη, αλλά πολλά τέτοια μικρά δέντρα μαζί μπορούν να δώσουν πολύ καλή πρόβλεψη.

Για παράδειγμα, ένα τέτοιο δέντρο μπορεί να κάνει κάτι τόσο απλό όσο:

«Αν το total sulfur dioxide είναι πάνω από κάποιο όριο, τότε γέρνω προς την κλάση white, αλλιώς γέρνω προς την κλάση red.»

Ένα μόνο τέτοιο split δεν μπορεί να περιγράψει όλη την πολυπλοκότητα των δεδομένων. Όμως το Boosting δεν βασίζεται σε ένα μόνο split. Προσθέτει 200 τέτοια μικρά κομμάτια πληροφορίας. Το καθένα είναι απλό, αλλά το άθροισμά τους μπορεί να γίνει πολύ αποτελεσματικό.

5.6 Γιατί χρησιμοποιούμε max_depth=1

Η εκφώνηση ζητά το κάθε επιμέρους δέντρο να έχει μέγιστο βάθος 1. Αυτό σημαίνει ότι κάθε δέντρο είναι πολύ απλό. Η επιλογή αυτή έχει νόημα στο Boosting για δύο λόγους.

Πρώτον, αν κάθε επιμέρους δέντρο ήταν πολύ βαθύ, τότε κάθε δέντρο θα μπορούσε να μάθει υπερβολικά πολλές λεπτομέρειες του training set. Αυτό θα αύξανε τον κίνδυνο overfitting, δηλαδή το μοντέλο να τα πηγαίνει πολύ καλά στα δεδομένα εκπαίδευσης αλλά όχι το ίδιο καλά σε νέα δεδομένα.

Δεύτερον, το Boosting θέλει συχνά πολλά μικρά και ελεγχόμενα βήματα. Ένα stump είναι ένα μικρό βήμα. Δεν προσπαθεί να λύσει όλο το πρόβλημα μόνο του. Απλώς προσθέτει μία μικρή διόρθωση στο συνολικό μοντέλο.

Άρα, το max_depth=1 κάνει κάθε επιμέρους μοντέλο απλό, αλλά επιτρέπει στο σύνολο των 200 μοντέλων να γίνει ισχυρό.

5.7 Τι σημαίνει learning rate

Η παράμετρος learning_rate ελέγχει πόσο έντονα επηρεάζει κάθε νέο μοντέλο το συνολικό ensemble.

Στο ερώτημα χρησιμοποιούμε:

$$\text{learning_rate} = 0.1.$$

Αυτό σημαίνει ότι κάθε νέο δέντρο προστίθεται με σχετικά μικρό βάρος. Με απλά λόγια, το μοντέλο μαθαίνει πιο αργά και πιο προσεκτικά. Ένα μικρό `learning_rate` μπορεί να βοηθήσει στη σταθερότητα της μεθόδου, επειδή αποφεύγει τις πολύ απότομες αλλαγές από ένα μόνο επιμέρους μοντέλο.

Ένας πρακτικός τρόπος να το καταλάβουμε είναι ο εξής:

- Αν το `learning_rate` ήταν μεγάλο, κάθε νέο δέντρο θα άλλαζε έντονα το συνολικό μοντέλο.
- Αν το `learning_rate` είναι μικρό, κάθε νέο δέντρο κάνει πιο προσεκτική διόρθωση.

Στο δικό μας μοντέλο το `learning_rate=0.1` σημαίνει ότι κάθε δέντρο δεν μπαίνει στο `ensemble` με πλήρη ένταση. Η συνεισφορά του μικραίνει. Έτσι το μοντέλο χρειάζεται αρκετά δέντρα, εδώ 200, αλλά η εκπαίδευση γίνεται πιο ομαλά.

5.8 Η σχέση ανάμεσα σε `n_estimators` και `learning_rate`

Οι δύο παράμετροι `n_estimators` και `learning_rate` πρέπει να διαβάζονται μαζί.

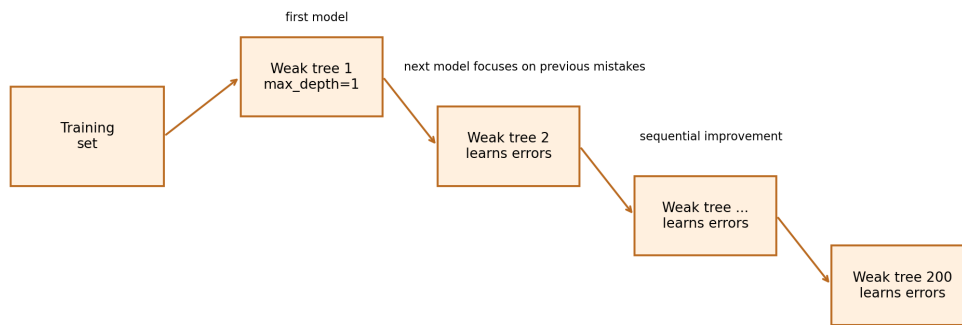
- Το `n_estimators=200` λέει πόσα μικρά δέντρα θα προστεθούν.
- Το `learning_rate=0.1` λέει πόσο δυνατά θα μετράει κάθε δέντρο.

Αν έχουμε μικρό `learning_rate`, συνήθως χρειαζόμαστε περισσότερα δέντρα για να μάθει αρκετά το μοντέλο. Αν έχουμε πολύ μεγάλο `learning_rate`, μπορεί το μοντέλο να μάθει γρήγορα, αλλά υπάρχει μεγαλύτερος κίνδυνος να κάνει απότομες και όχι τόσο σταθερές διορθώσεις.

Στην εργασία, οι τιμές δεν επιλέγονται από εμάς ελεύθερα σε αυτό το ερώτημα. Μας δίνονται από την εκφώνηση:

```
n_estimators=200, learning_rate=0.1, max_depth=1.
```

Αρα εδώ εφαρμόζουμε ακριβώς το μοντέλο που ζητάει η εκφώνηση. Η διερεύνηση διαφορετικών τιμών θα γίνει αργότερα στο `tuning`.



Boosting builds trees sequentially. Each new weak tree tries to improve the current ensemble.

Σχήμα 9: Η βασική ιδέα του Boosting: τα επιμέρους μοντέλα χτίζονται διαδοχικά.

5.9 Πώς παίρνει τελική απόφαση το Boosting

Στο Bagging και στο Random Forest λέγαμε ότι τα δέντρα ψηφίζουν. Στο Gradient Boosting η εικόνα είναι λίγο διαφορετική. Τα μικρά δέντρα προσθέτουν διαδοχικά διορθώσεις σε ένα συνολικό score.

Μπορούμε να το σκεφτούμε απλά ως εξής:

1. Το μοντέλο ξεκινά με μία αρχική εκτίμηση.
2. Το πρώτο μικρό δέντρο προσθέτει μία μικρή διόρθωση.
3. Το δεύτερο μικρό δέντρο προσθέτει άλλη μία μικρή διόρθωση.
4. Μετά από 200 τέτοιες διορθώσεις, το μοντέλο έχει ένα τελικό score.
5. Το score μετατρέπεται σε πρόβλεψη κλάσης, δηλαδή red ή white.

Αρα δεν είναι απλώς «200 ψήφοι». Είναι περισσότερο σαν άθροισμα πολλών μικρών διορθώσεων. Αυτή είναι η βασική λογική του Gradient Boosting.

5.10 Πώς το λύσαμε

Χρησιμοποιήσαμε το ίδιο dataset, τους ίδιους predictors και το ίδιο train/test split με τα προηγούμενα ερωτήματα. Η μεταβλητή στόχος ήταν πάλι η wine_type, με κωδικοποίηση red=0 και white=1.

Το μοντέλο ορίστηκε ως:

```
model = GradientBoostingClassifier(  
    n_estimators=200,  
    learning_rate=0.1,  
    max_depth=1,  
    random_state=6931,  
)
```

Ας δούμε τι σημαίνει κάθε παράμετρος:

- `GradientBoostingClassifier`: δηλώνει ότι χρησιμοποιούμε Boosting για ταξινόμηση.
- `n_estimators=200`: το μοντέλο θα αποτελείται από 200 επιμέρους δέντρα.
- `learning_rate=0.1`: κάθε νέο δέντρο επηρεάζει το συνολικό μοντέλο με σχετικά μικρό βήμα.
- `max_depth=1`: κάθε επιμέρους δέντρο είναι πολύ απλό.
- `random_state=6931`: τα αποτελέσματα είναι αναπαραγώγιμα.

Η συνολική μεθοδολογία ήταν:

1. Φορτώσαμε τα δεδομένα κρασιού.
2. Αφαιρέσαμε τη μεταβλητή `quality`, επειδή η εκφώνηση λέει να μη χρησιμοποιηθεί ως `predictor`.
3. Ορίσαμε ως στόχο τη μεταβλητή `wine_type`.
4. Χωρίσαμε τα δεδομένα σε `training set` 70% και `test set` 30%, όπως και στα προηγούμενα ερωτήματα.
5. Εκπαιδεύσαμε το `GradientBoostingClassifier` μόνο στο `training set`.
6. Χρησιμοποιήσαμε το εκπαιδευμένο μοντέλο για να προβλέψουμε τις κλάσεις των παρατηρήσεων του `test set`.
7. Συγκρίναμε τις προβλέψεις με τις πραγματικές τιμές του `test set`.
8. Υπολογίσαμε το `test accuracy`, το `test error` και τον `confusion matrix`.

Είναι σημαντικό ότι το `test set` δεν χρησιμοποιείται για την εκπαίδευση. Το κρατάμε μόνο για την τελική αξιολόγηση, ώστε να δούμε πώς συμπεριφέρεται το μοντέλο σε δεδομένα που δεν έχει δει.

Μετά την εκπαίδευση, έγιναν προβλέψεις στο `test set`:

```
y_pred = model.predict(X_test)
```

και υπολογίστηκαν:

```
accuracy = accuracy_score(y_test, y_pred)  
test_error = 1 - accuracy
```

5.11 Αποτελέσματα

Πίνακας 13: Αποτελέσματα Boosting.

Μέτρο	Τιμή
n_estimators	200
learning_rate	0.1
max_depth	1
Test accuracy	0.9679
Test error	0.0321
Σωστές προβλέψεις	1538
Λανθασμένες προβλέψεις	51

Το Boosting πέτυχε:

$$\text{Test accuracy} = 0.9679$$

δηλαδή ταξινόμησε σωστά περίπου το 96.79% των test παρατηρήσεων.

Αν το γράψουμε ως ποσοστά:

$$0.9679 \times 100 = 96.79\%.$$

Το αντίστοιχο test error είναι:

$$1 - 0.9679 = 0.0321,$$

δηλαδή περίπου:

$$0.0321 \times 100 = 3.21\%.$$

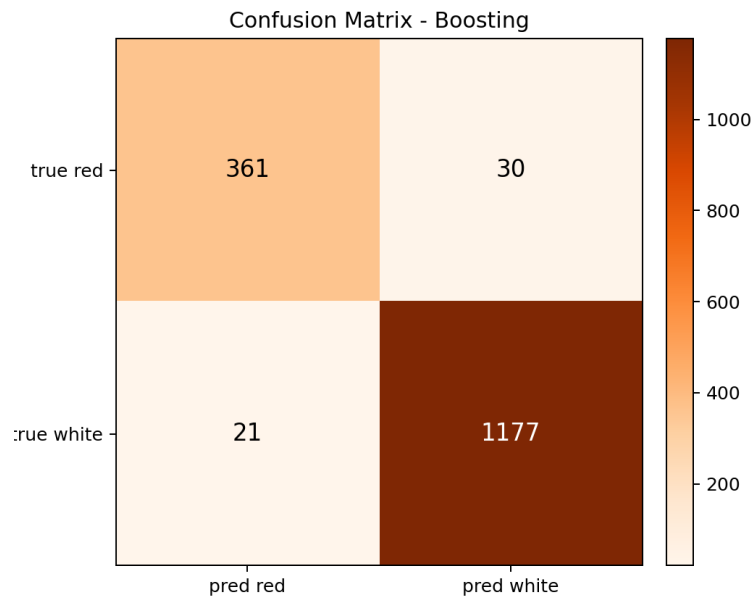
Άρα, σε απλή γλώσσα, το μοντέλο κάνει λάθος περίπου σε 3 από τις 100 παρατηρήσεις του test set.

5.12 Confusion matrix

Ο confusion matrix ήταν:

Πίνακας 14: Confusion matrix για το Boosting.

	Πρόβλεψη red	Πρόβλεψη white
Πραγματικό red	361	30
Πραγματικό white	21	1177



Σχήμα 10: Confusion matrix του Boosting.

Ο πίνακας δείχνει ότι:

- 361 κόκκινα κρασιά ταξινομήθηκαν σωστά ως κόκκινα,
- 30 κόκκινα κρασιά ταξινομήθηκαν λάθος ως λευκά,
- 21 λευκά κρασιά ταξινομήθηκαν λάθος ως κόκκινα,
- 1177 λευκά κρασιά ταξινομήθηκαν σωστά ως λευκά.

Άρα οι σωστές προβλέψεις είναι:

$$361 + 1177 = 1538.$$

Τα συνολικά λάθη είναι:

$$30 + 21 = 51.$$

Παρατηρούμε επίσης ότι το μοντέλο έκανε:

- 30 λάθη όπου κόκκινα κρασιά προβλέφθηκαν ως λευκά,
- 21 λάθη όπου λευκά κρασιά προβλέφθηκαν ως κόκκινα.

Δηλαδή, στο συγκεκριμένο test set, το Boosting μπερδεψε λίγο περισσότερα κόκκινα κρασιά προς την κατεύθυνση του λευκού από ό,τι λευκά προς την κατεύθυνση του κόκκινου. Παρ' όλα αυτά, το συνολικό πλήθος λαθών είναι πολύ μικρό.

5.13 Σύγκριση με τα προηγούμενα μοντέλα

Πίνακας 15: Σύγκριση των τεσσάρων πρώτων μοντέλων.

Μοντέλο	Test Accuracy	Test Error
Decision Tree (max_depth=2)	0.9339	0.0661
Bagging (200 estimators)	0.9660	0.0340
Random Forest (200 estimators, $m = 5$)	0.9666	0.0334
Boosting (200 estimators)	0.9679	0.0321

Μέχρι τώρα, το Boosting έχει την καλύτερη test accuracy. Η διαφορά από το Random Forest είναι μικρή, αλλά το Boosting έκανε τα λιγότερα συνολικά λάθη:

Μοντέλο	Λάθη	Σωστές προβλέψεις
Decision Tree	105	1484
Bagging	54	1535
Random Forest	53	1536
Boosting	51	1538

Η διαφορά ανάμεσα σε Bagging, Random Forest και Boosting δεν είναι τεράστια, επειδή και οι τρεις ensemble μέθοδοι δουλεύουν πολύ καλά στο συγκεκριμένο πρόβλημα. Όμως το Boosting έχει το μικρότερο test error από τα αρχικά μοντέλα:

$$0.0321 < 0.0334 < 0.0340 < 0.0661.$$

Αυτό σημαίνει ότι, στο συγκεκριμένο train/test split, το Boosting ήταν το πιο ακριβές από τα τέσσερα πρώτα μοντέλα.

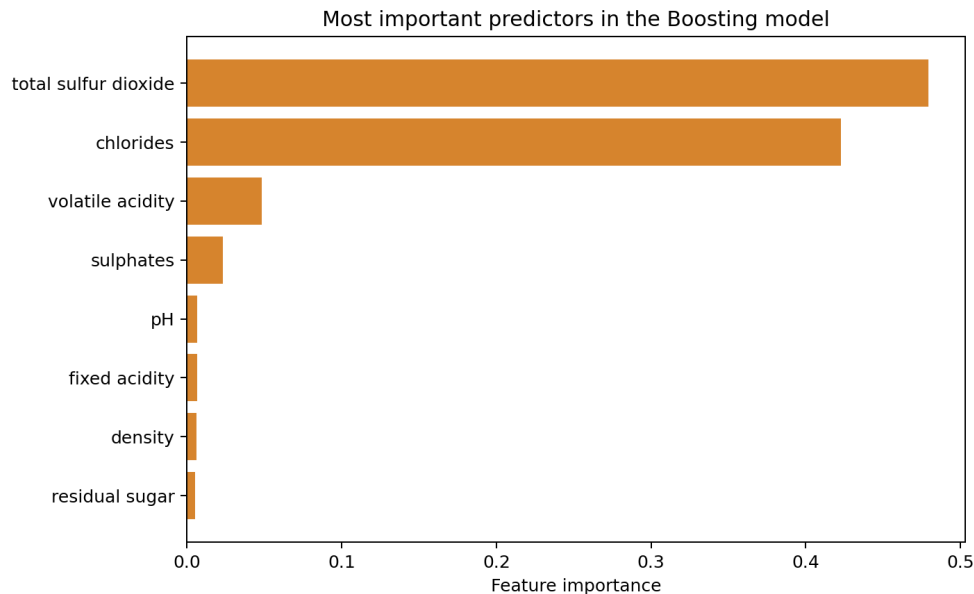
Η βελτίωση σε σχέση με το Random Forest είναι μικρή:

$$0.9679 - 0.9666 = 0.0013.$$

Σε ποσοστιαίες μονάδες, αυτό είναι περίπου 0.13 ποσοστιαίες μονάδες. Άρα δεν λέμε ότι το Boosting είναι εντυπωσιακά καλύτερο από το Random Forest. Λέμε ότι ήταν οριακά καλύτερο στα συγκεκριμένα test δεδομένα.

5.14 Σημαντικότητα χαρακτηριστικών

Το Boosting δίνει επίσης μία ένδειξη για το ποιες μεταβλητές βοήθησαν περισσότερο στην ταξινόμηση.



Σχήμα 11: Οι σημαντικότερες μεταβλητές στο Boosting.

Οι δύο σημαντικότερες μεταβλητές ήταν και εδώ οι:

- total sulfur dioxide
- chlorides

Αυτό επιβεβαιώνει για ακόμη μία φορά ότι αυτές οι μεταβλητές είναι πολύ χρήσιμες για τη διάκριση λευκού και κόκκινου κρασιού.

Η ερμηνεία εδώ είναι παρόμοια με τα προηγούμενα δέντρα, αλλά χρειάζεται λίγη προσοχή. Η σημαντικότητα χαρακτηριστικών στο Boosting προκύπτει από το πόσο συχνά και πόσο αποτελεσματικά χρησιμοποιείται μία μεταβλητή στα μικρά δέντρα για να μειώσει το λάθος του μοντέλου.

Επειδή κάθε stump έχει μόνο ένα split, κάθε μικρό δέντρο μπορεί να βασιστεί σε μία βασική μεταβλητή κάθε φορά. Αν μία μεταβλητή εμφανίζεται συχνά σε χρήσιμα splits, τότε αποκτά μεγαλύτερη σημαντικότητα.

Το ότι οι total sulfur dioxide και chlorides παραμένουν σημαντικές και στο Boosting είναι σημαντικό εύρημα, γιατί βλέπουμε συνέπεια ανάμεσα σε διαφορετικές μεθόδους:

- το απλό δέντρο τις χρησιμοποίησε στους βασικούς διαχωρισμούς,
- το Bagging τις ανέδειξε ως ισχυρές μεταβλητές,
- το Random Forest τις ανέδειξε ξανά,
- και το Boosting δείχνει ότι βοηθούν και στη διαδοχική διόρθωση των προβλέψεων.

5.15 Συμπέρασμα Ερωτήματος 4

Το Boosting με 200 επιμέρους μοντέλα, `learning_rate=0.1` και `max_depth=1` πέτυχε `test accuracy 0.9679`. Είναι το καλύτερο αποτέλεσμα μέχρι στιγμής ανάμεσα στα αρχικά μοντέλα των Ερωτημάτων 1-4.

Η μέθοδος πέτυχε υψηλή απόδοση παρότι κάθε επιμέρους δέντρο ήταν πολύ απλό. Αυτό δείχνει τη δύναμη του Boosting: πολλά απλά μοντέλα, όταν συνδυαστούν διαδοχικά και σωστά, μπορούν να δημιουργήσουν έναν πολύ ισχυρό ταξινομητή.

Το βασικό συμπέρασμα είναι ότι το Boosting λειτουργεί καλά επειδή μαθαίνει σταδιακά. Δεν προσπαθεί να φτιάξει ένα πολύ περίπλοκο δέντρο από την αρχή. Αντίθετα, φτιάχνει πολλά απλά δέντρα, όπου το καθένα διορθώνει λίγο το συνολικό μοντέλο. Με `learning_rate=0.1`, αυτές οι διορθώσεις είναι προσεκτικές, και με `n_estimators=200` υπάρχουν αρκετά βήματα ώστε το μοντέλο να γίνει πολύ ακριβές.

6 Ερώτημα 5: Test error του δέντρου συναρτήσει του βάθους

6.1 Τι ζητάει το ερώτημα

Το πέμπτο ερώτημα είναι:

«Να παρασταθεί γραφικά το test error της μεθόδου του ερωτήματος 1 συναρτήσει του βάθους του δέντρου. Τι παρατηρείτε; Σχολιάστε.»

Με απλά λόγια, μέχρι τώρα στο Ερώτημα 1 χρησιμοποιήσαμε ένα δέντρο με `max_depth=2`. Τώρα η εκφώνηση μάς ζητά να μην κρατήσουμε σταθερό το βάθος, αλλά να δοκιμάσουμε πολλά διαφορετικά βάθη και να δούμε πώς αλλάζει το test error.

Το ερώτημα θέλει να απαντήσουμε σε κάτι πολύ σημαντικό:

Αν κάνουμε το δέντρο πιο βαθύ, γίνεται πάντα καλύτερο;

Η απάντηση, όπως θα δούμε, είναι όχι. Μέχρι ένα σημείο το μεγαλύτερο βάθος βοηθά. Μετά από ένα σημείο, όμως, το δέντρο γίνεται υπερβολικά σύνθετο και μπορεί να αρχίσει να γενικεύει χειρότερα.

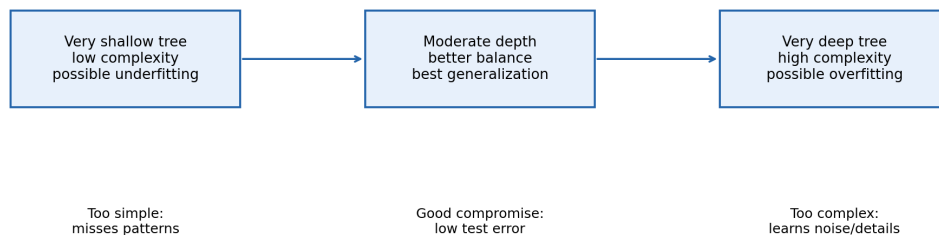
6.2 Θεωρητικό υπόβαθρο

Το βάθος ενός δέντρου καθορίζει πόσο σύνθετο μπορεί να γίνει το μοντέλο.

- Ένα πολύ ρηχό δέντρο έχει λίγους κανόνες και είναι πολύ απλό.
- Ένα πολύ βαθύ δέντρο έχει πολλούς κανόνες και μπορεί να ξεχωρίσει λεπτομέρειες στα δεδομένα.

Αν το δέντρο είναι υπερβολικά απλό, μπορεί να μην καταλαβαίνει αρκετά καλά τη σχέση ανάμεσα στα χαρακτηριστικά και στον τύπο κρασιού. Αυτό λέγεται **underfitting**.

Αν το δέντρο είναι υπερβολικά βαθύ, μπορεί να αρχίσει να μαθαίνει όχι μόνο το γενικό μοτίβο, αλλά και τυχαίες λεπτομέρειες του training set. Αυτό λέγεται **overfitting**.



Question 5 checks how tree complexity affects test error.

Σχήμα 12: Η βασική ιδέα του Ερωτήματος 5: πολύ μικρό, μέτριο και πολύ μεγάλο βάθος.

6.3 Τι είναι test error

Το test error είναι:

$$\text{Test Error} = 1 - \text{Test Accuracy}.$$

Αν ένα μοντέλο έχει test accuracy 0.9339, τότε:

$$\text{Test Error} = 1 - 0.9339 = 0.0661.$$

Το test error δείχνει το ποσοστό των test παρατηρήσεων που ταξινομήθηκαν λάθος. Όσο μικρότερο είναι, τόσο καλύτερο είναι το μοντέλο.

6.4 Πώς το λύσαμε

Χρησιμοποιήσαμε το ίδιο dataset και το ίδιο train/test split με τα προηγούμενα ερωτήματα. Στη συνέχεια εκπαιδεύσαμε πολλά δέντρα ταξινόμησης, αλλάζοντας την τιμή του max_depth.

Δοκιμάσαμε βάθη:

$$\text{max_depth} = 1, 2, 3, \dots, 20.$$

Για κάθε βάθος εκπαιδεύτηκε ξεχωριστό δέντρο και υπολογίστηκε το test error:

```

for depth in range(1, 21):
    model = DecisionTreeClassifier(max_depth=depth, random_state
    =6931)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    test_error = 1 - accuracy

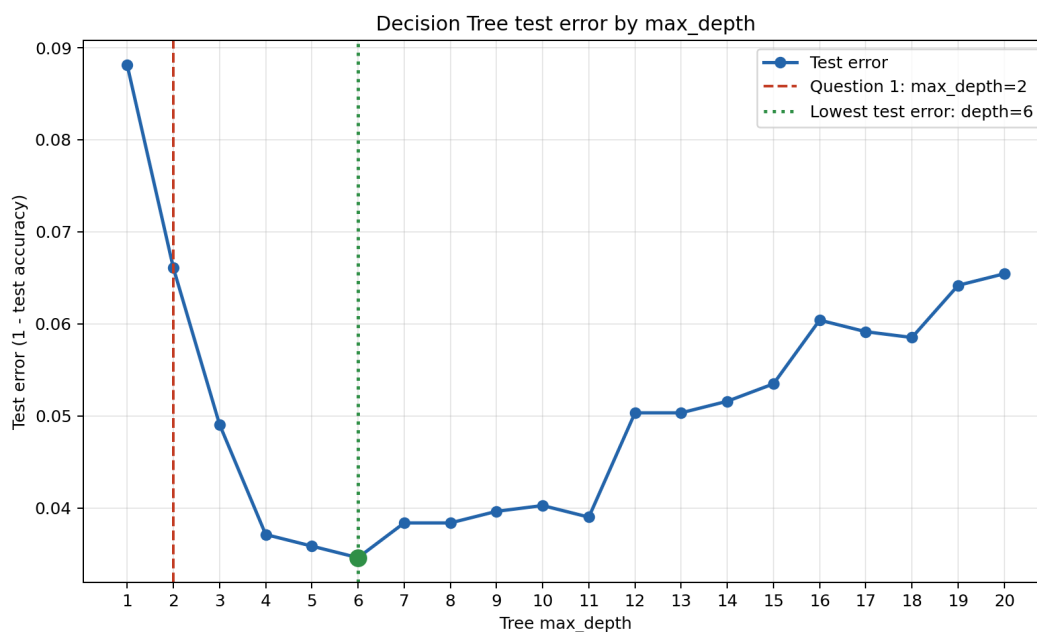
```

Έτσι πήραμε έναν πίνακα που δείχνει πώς αλλάζει το test error ανάλογα με το βάθος του δέντρου.

6.5 Αριθμητικά αποτελέσματα

Πίνακας 16: Test accuracy και test error για διαφορετικά βάθη δέντρου.

Depth	Accuracy	Error	Depth	Accuracy	Error
1	0.9119	0.0881	11	0.9610	0.0390
2	0.9339	0.0661	12	0.9497	0.0503
3	0.9509	0.0491	13	0.9497	0.0503
4	0.9629	0.0371	14	0.9484	0.0516
5	0.9641	0.0359	15	0.9465	0.0535
6	0.9654	0.0346	16	0.9396	0.0604
7	0.9616	0.0384	17	0.9408	0.0592
8	0.9616	0.0384	18	0.9415	0.0585
9	0.9604	0.0396	19	0.9358	0.0642
10	0.9597	0.0403	20	0.9346	0.0654



Σχήμα 13: Test error του δέντρου ταξινόμησης συναρτήσει του max_depth.

6.6 Τι παρατηρούμε

Στα πολύ μικρά βάθη, το test error είναι σχετικά υψηλό. Για παράδειγμα:

- για `max_depth=1`, το test error είναι 0.0881,
- για `max_depth=2`, το test error είναι 0.0661.

Αυτό σημαίνει ότι ένα πολύ απλό δέντρο δεν αρκεί για να αποτυπώσει πλήρως τη δομή των δεδομένων. Έχει λίγους κανόνες και άρα δεν μπορεί να διαχωρίσει τέλεια τα λευκά από τα κόκκινα κρασιά.

Καθώς αυξάνεται το βάθος, το test error μειώνεται. Η καλύτερη τιμή εμφανίζεται στο:

$$\text{max_depth} = 6.$$

Για αυτό το βάθος έχουμε:

$$\text{Test Accuracy} = 0.9654, \quad \text{Test Error} = 0.0346.$$

Άρα το δέντρο βάθους 6 είναι πολύ καλύτερο από το αρχικό δέντρο βάθους 2. Συγκεκριμένα, το test error μειώνεται από 0.0661 σε 0.0346.

6.7 Γιατί μετά το βάθος 6 χειροτερεύει

Μετά το βάθος 6, το test error δεν συνεχίζει να μειώνεται. Αντίθετα, αρχίζει γενικά να αυξάνεται. Για παράδειγμα:

- στο βάθος 10, το test error είναι 0.0403,
- στο βάθος 15, το test error είναι 0.0535,
- στο βάθος 20, το test error είναι 0.0654.

Αυτό είναι ένδειξη *overfitting*. Το βαθύτερο δέντρο μαθαίνει όλο και πιο λεπτομερείς κανόνες από το training set. Αυτοί οι κανόνες μπορεί να ταιριάζουν καλά στα training δεδομένα, αλλά να μην είναι τόσο χρήσιμοι για νέα δεδομένα.

Με άλλα λόγια:

Το πιο σύνθετο μοντέλο δεν είναι πάντα το καλύτερο μοντέλο.

6.8 Συμπέρασμα Ερωτήματος 5

Το test error μειώνεται καθώς το βάθος αυξάνεται από 1 έως 6, επειδή το δέντρο αποκτά περισσότερη ευελιξία και μπορεί να περιγράψει καλύτερα τη δομή των δεδομένων. Η καλύτερη τιμή εμφανίζεται στο `max_depth=6`, με test accuracy 0.9654 και test error 0.0346.

Για μεγαλύτερα βάθη, το test error αρχίζει να αυξάνεται. Αυτό δείχνει ότι το δέντρο γίνεται υπερβολικά σύνθετο και πιθανόν υπερπροσαρμόζεται στα training δεδομένα. Επομένως, ένα μέτριο βάθος δέντρου προσφέρει την καλύτερη ισορροπία ανάμεσα σε απλότητα και προβλεπτική ικανότητα.

7 Συγκεντρωτική εικόνα μέχρι το Ερώτημα 5

Πίνακας 17: Συγκεντρωτικός πίνακας αποτελεσμάτων μέχρι το Ερώτημα 5.

Ερώτημα	Μέθοδος	Accuracy	Error
1	Decision Tree	0.9339	0.0661
2	Bagging	0.9660	0.0340
3	Random Forest	0.9666	0.0334
4	Boosting	0.9679	0.0321
5	Best Tree Depth (6)	0.9654	0.0346

Μέχρι στιγμής, το Boosting έχει την καλύτερη ακρίβεια ανάμεσα στα αρχικά μοντέλα των Ερωτημάτων 1-4. Το Ερώτημα 5 δείχνει όμως ότι το αρχικό δέντρο βάθους 2 δεν ήταν το καλύτερο δυνατό απλό δέντρο. Αν αυξήσουμε το βάθος μέχρι το 6, το δέντρο φτάνει σε test accuracy 0.9654, δηλαδή πλησιάζει πολύ τις ensemble μεθόδους.

Α' Αρχεία που χρησιμοποιούνται στο ενιαίο explanatory report

Τα βασικά βοηθητικά αρχεία που έχουν δημιουργηθεί μέχρι τώρα είναι:

- q1_explanatory_decision_tree.py
- q1_root_split_feature_candidates.csv
- q2_explanatory_bagging.py
- q3_explanatory_random_forest.py
- q4_explanatory_boosting.py
- q5_explanatory_tree_depth.py
- q1_confusion_matrix.png
- q1_decision_tree_plot.png
- q2_confusion_matrix.png
- q2_bagging_concept.png
- q2_bagging_feature_importance.png
- q3_confusion_matrix.png
- q3_random_forest_concept.png
- q3_random_forest_feature_importance.png
- q4_confusion_matrix.png
- q4_boosting_concept.png

- q4_boosting_feature_importance.png
- q5_tree_depth_concept.png
- q5_tree_depth_error.png